JAKARTA EE

# CHEAT SHEETS

*A Complete Reference for Jakarta EE 11 Enterprise Java*

Platform & Profiles · JavaBeans · Enterprise Beans · CDI · Servlets · Faces · WebSocket · REST & JSON · Persistence · Jakarta Data · Transactions · Validation · Security · Concurrency · Messaging · MicroProfile

**40** REFERENCE SHEETS

CONTENTS


**PLATFORM & SETUP**

01 Introduction & the Platform
02 javax → jakarta & Version History
03 Runtimes & Project Setup
04 App Structure & Deployment

**JAVABEANS & BEAN MODELS**

05 JavaBeans Conventions
06 Introspection & Property Change Events
07 Which Bean Is Which?

**ENTERPRISE BEANS**

08 Stateless & Singleton Session Beans
09 Stateful Beans, Lifecycle & Timers
10 Enterprise Beans vs CDI Beans

**CDI**

11 CDI Basics & Injection
12 Scopes & Lifecycle
13 Qualifiers, Producers & Alternatives
14 Events, Interceptors & Decorators

**WEB TIER**

15 Servlets
16 Filters, Listeners & Sessions
17 Pages & Expression Language
18 Faces Basics
19 Faces Navigation, Validation & AJAX
20 WebSocket

**REST & JSON**

21 REST Resources
22 Parameters, Responses & Errors
23 JSON Processing & Binding
24 REST Client, Filters & Providers

**PERSISTENCE & DATA**

25 Entities & persistence.xml
26 Relationships & Inheritance
27 EntityManager & Entity Lifecycle
28 JPQL, Criteria & Fetching
29 Jakarta Data Repositories
30 Transactions

**VALIDATION, SECURITY & MESSAGING**

31 Bean Validation
32 Security
33 Concurrency
34 Messaging
35 Batch, Mail & Other Specs

**TESTING & PRODUCTION**

36 Testing Jakarta Apps
37 MicroProfile
38 Packaging, Containers & What's Next

**QUICK REFERENCE**

39 Quick Reference: Annotations
40 Quick Reference: Specs, Versions & Files

JEE

# Table of Contents

40 sheets · syntax · examples · notes


## PLATFORM & SETUP 01 – 04

- 01 Introduction & the Plat…
- 02 javax → jakarta & Versi…
- 03 Runtimes & Project Se…
- 04 App Structure & Deplo…

## JAVABEANS & BEAN MODELS 05 – 07

- 05 JavaBeans Conventions
- 06 Introspection & Proper…
- 07 Which Bean Is Which?

## ENTERPRISE BEANS 08 – 10

- 08 Stateless & Singleton S…
- 09 Stateful Beans, Lifecyc…
- 10 Enterprise Beans vs C…

## CDI 11 – 14

- 11 CDI Basics & Injection
- 12 Scopes & Lifecycle
- 13 Qualifiers, Producers &…
- 14 Events, Interceptors & …

## WEB TIER 15 – 20

- 15 Servlets
- 16 Filters, Listeners & Ses…
- 17 Pages & Expression La…
- 18 Faces Basics
- 19 Faces Navigation, Valid…
- 20 WebSocket

## REST & JSON 21 – 24

- 21 REST Resources
- 22 Parameters, Response…
- 23 JSON Processing & Bin…
- 24 REST Client, Filters & P…

## PERSISTENCE & DATA 25 – 30

- 25 Entities & persistence.…
- 26 Relationships & Inherit…
- 27 EntityManager & Entit…
- 28 JPQL, Criteria & Fetchi…
- 29 Jakarta Data Repositor…
- 30 Transactions

## VALIDATION, SECURITY & MESSAGING 31 – 35

- 31 Bean Validation
- 32 Security
- 33 Concurrency
- 34 Messaging
- 35 Batch, Mail & Other Sp…

## TESTING & PRODUCTION 36 – 38

- 36 Testing Jakarta Apps
- 37 MicroProfile
- 38 Packaging, Containers …

## QUICK REFERENCE 39 – 40

- 39 Quick Reference: Anno…
- 40 Quick Reference: Spec…

# Introduction & the Platform



## DESCRIPTION

**Jakarta EE** is a set of **specifications** for server-side Java: CDI, Servlet, REST, Persistence, Security and more. A spec defines the API (`jakarta.*` interfaces and annotations); a **compatible implementation** (WildFly, Payara, Open Liberty, GlassFish) provides the code. You write against the API and deploy to any compatible runtime. The Eclipse Foundation runs it; it was Java EE under Oracle until 2017.

## NOTES

- Targets **Jakarta EE 11** (Java 17+, Java 21 for virtual threads)
- Three profiles: **Core** ⊂ **Web** ⊂ **Platform**
- Spring Boot runs on the same `jakarta.*` APIs (Servlet, JPA, Validation)

## A COMPLETE JAKARTA REST + CDI APP

```
@ApplicationPath("/api")                    // turns on Jakarta REST
public class App extends Application { }
@ApplicationScoped                          // CDI bean, one per app
public class GreetingService {
    public String greet(String name) { return "Hello, " + name; }
}

@Path("/hello")                             // GET /api/hello/Bob
public class HelloResource {
    @Inject GreetingService service;        // container injects it

    @GET @Path("{name}")
    @Produces(MediaType.TEXT_PLAIN)
    public String hello(@PathParam("name") String name) {
        return service.greet(name);         // Hello, Bob
    }
}
```

## CODE EXPLAINED

- → `@ApplicationPath` — base URI for every REST resource in the war
- → `@ApplicationScoped` — one shared instance, created lazily by CDI
- → `@Inject` — the container supplies the bean; no `new`
- → No `main()` — the server owns the lifecycle and the threads

## SPRING EQUIVALENT

- Jakarta REST `@Path` → `@RestController` + `@RequestMapping`
- CDI `@ApplicationScoped` → singleton `@Service`
- Spring uses Jakarta Servlet, JPA, Validation directly · See Spring 01

## PREFER / AVOID

- ⇄ ~~`javax.ws.rs.*`~~ imports → `jakarta.ws.rs.*` (EE 9+)
- ⇄ fat war bundling the API → `jakartaee-api` as `provided`
- ⇄ picking a spec per tutorial → check the EE 11 spec list (sheet 40)

## PROFILES

| PROFILE | ADDS | FOR |
|---|---|---|
| `Core` | CDI Lite, REST, JSON-P/B, Interceptors | microservices, native images |
| `Web` | + Servlet, Faces, Persistence, Security, Data | typical web apps |
| `Platform` | + Messaging, Batch, Mail, EJB full, Connectors | full enterprise apps |

## SPEC VS IMPLEMENTATION

| SPEC | IMPLEMENTATIONS |
|---|---|
| `CDI 4.1` | Weld, OpenWebBeans |
| `Persistence 3.2` | Hibernate, EclipseLink |
| `REST 4.0` | RESTEasy, Jersey |
| `Faces 4.1` | Mojarra, MyFaces |

## ⚠ COMMON MISTAKES

- Mixing `javax.*` and `jakarta.*` jars — classes don't match and injection silently fails
- Treating the spec as a product — features beyond the spec are vendor extensions and don't port
- Choosing Platform when Web is enough — bigger server, slower start, more to secure

## ✓ TIPS & BEST PRACTICES

- Start from **start.jakarta.ee** to get a working war for your runtime
- Read the spec's own docs at `jakarta.ee/specifications` for exact rules
- Keep vendor-specific config in one place so switching runtimes stays cheap

## QUICK SUMMARY

- `spec` — API contract; runtime supplies the code
- `Core / Web / Platform` — nested profiles
- `jakarta.*` — namespace since EE 9
- `EE 11` — Java 17+, Data 1.0, Concurrency 3.1

# javax → jakarta & Version History



## DESCRIPTION

Oracle gave Java EE to Eclipse in 2017 but kept the `javax` name. **Jakarta EE 8** was the same API under a new name; **EE 9** renamed every package from `javax.*` to `jakarta.*`, which is the one breaking change most migrations are about. EE 10 and 11 then added features and raised the Java baseline. Libraries must match: a `javax` library won't see `jakarta` annotations.

## NOTES

- Java SE `javax.*` packages (`javax.sql`, `javax.crypto`) did **not** move
- EE 11 removed Managed Beans and the SecurityManager hooks
- Spring Boot 3+ = `jakarta.*`; Boot 2 = `javax.*`
- Hibernate 6+, Tomcat 10+, Jetty 11+ are `jakarta` only

## SAME CLASS, BEFORE AND AFTER EE 9

```
// Java EE 8 / Jakarta EE 8
import javax.inject.Inject;
import javax.persistence.Entity;
import javax.ws.rs.GET;

// Jakarta EE 9, 10, 11
import jakarta.inject.Inject;
import jakarta.persistence.Entity;
import jakarta.ws.rs.GET;
```

## MIGRATE WITH OPENREWRITE (MAVEN)

```
mvn -U org.openrewrite.maven:rewrite-maven-plugin:run \
  -Drewrite.recipeArtifactCoordinates=\
org.openrewrite.recipe:rewrite-migrate-java:RELEASE \
  -Drewrite.activeRecipes=\
org.openrewrite.java.migrate.jakarta.JakartaEE10
```

## CODE EXPLAINED

→ Only the **package prefix** changes; class names stay

→ OpenRewrite also fixes `persistence.xml`, `web.xml` and dependency coordinates

→ **Eclipse Transformer** rewrites compiled jars you can't rebuild

## SPRING EQUIVALENT

- Boot 2 → 3 is the same `javax` → `jakarta` move
- Spring's own packages (`org.springframework`) never changed · See Spring 01

## PREFER / AVOID

⇄ find-and-replace ~~`javax.`~~ → a recipe that skips Java SE `javax.*`

⇄ staying on EE 8 for libraries → upgrade or shade the library

⇄ old XML schema URLs → `jakarta.ee/xml/ns/jakartaee` namespaces

## VERSION HISTORY

| RELEASE | YEAR | KEY CHANGE |
|---|---|---|
| Java EE 8 | 2017 | last Oracle release, `javax` |
| Jakarta EE 8 | 2019 | same API, Eclipse governance |
| Jakarta EE 9 | 2020 | `javax` → `jakarta` rename |
| Jakarta EE 10 | 2022 | Core Profile, Java 11+, CDI Lite |
| Jakarta EE 11 | 2025 | Java 17+, Data 1.0, records |
| Jakarta EE 12 | 2026–27 | in milestones; consistency, Query spec |

## REMOVED OR PRUNED

| SPEC | STATUS |
|---|---|
| Managed Beans | removed EE 11 — use CDI |
| SecurityManager | removed EE 11 |
| XML Web Services | optional since EE 9 |
| JAX-RPC, EJB entity beans | gone since EE 7/8 |
| Deployment API | removed EE 9 |

## ⚠ COMMON MISTAKES

- Renaming Java SE packages too — `javax.sql.DataSource` must stay `javax`
- Upgrading the server but not a library — e.g. an old Jersey filter compiled against `javax.ws.rs`
- Forgetting XML descriptors — a `javax` namespace in `beans.xml` is ignored

## ✓ TIPS & BEST PRACTICES

- Run OpenRewrite on a branch, then build and test before touching anything else
- `mvn dependency:tree | grep javax` finds stragglers
- IntelliJ: Refactor › Migrate Packages and Classes › Java EE to Jakarta EE

## QUICK SUMMARY

- `EE 9` — the package rename
- `EE 10` — Core Profile, CDI Lite
- `EE 11` — Java 17, Jakarta Data
- `OpenRewrite` — automated migration

Sheet 03 · Platform API 11.0

# Runtimes & Project Setup

WildFly · Payara · Open Liberty · GlassFish · `jakarta.jakartaee-api` · `provided` · war

## DESCRIPTION

A Jakarta EE app is usually a **war** that runs inside a **compatible runtime**. The server already contains every spec implementation, so your build depends on the API jar with `provided` scope: you compile against it but never ship it. Pick the Web or Platform API to match your runtime's profile.

## NOTES

- `jakartaee-api` = Platform · `jakartaee-web-api` = Web · `jakartaee-core-api` = Core
- Runtimes list their EE 11 certification on jakarta.ee/compatibility
- Open Liberty and Payara Micro also run as an executable jar
- Gradle: `compileOnly` is the same as Maven's `provided`

## UNDER THE CONTAINER

- The server provides the API jars — `provided` keeps a second copy out of the war
- Each app gets its own classloader; server classes load first for `jakarta.*`
- Bundling a spec jar loads two copies of a class → `ClassCastException` on the same type name

## POM.XML — THE MINIMAL JAKARTA EE 11 WAR

```
<packaging>war</packaging>
<properties>
  <maven.compiler.release>21</maven.compiler.release>
</properties>
<dependencies>
  <dependency>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakarta.jakartaee-api</artifactId>
    <version>11.0.0</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
<build><finalName>shop</finalName></build>
```

## CODE EXPLAINED

- → `war` — web archive the server deploys
- → `jakarta.jakartaee-api` — all Platform APIs, compile only
- → `provided` — on the classpath at build time, not in `WEB-INF/lib`
- → `finalName` — `shop.war`, default context root `/shop`

## SPRING EQUIVALENT

- Spring Boot ships the server inside the jar (embedded Tomcat)
- `spring-boot-starter-*` replaces the API jar · See Spring 03

## PREFER / AVOID

- ~~`compile`~~ scope on the API → `provided`
- ~~pinning each spec version~~ → one `jakartaee-api` version
- ~~copying a 2015 pom~~ → generate one at start.jakarta.ee

## RUNTIMES

| RUNTIME | VENDOR | NOTES |
|---|---|---|
| WildFly | Red Hat | Platform + Web; Galleon trims layers |
| Payara | Payara | GlassFish fork; Micro jar |
| Open Liberty | IBM | feature-based, fast dev mode |
| GlassFish | Eclipse | reference implementation |
| TomEE | Apache | Tomcat + Web Profile |

## DEV COMMANDS

| COMMAND | DOES |
|---|---|
| `mvn package` | builds `target/shop.war` |
| `mvn liberty:dev` | Open Liberty hot reload |
| `mvn wildfly:dev` | WildFly hot reload |
| `mvn payara-micro:dev` | Payara Micro dev loop |

## ⚠ COMMON MISTAKES

- Using `compile` scope — the war carries API jars that clash with the server's
- Compiling for Java 21 but running the server on 17 — `UnsupportedClassVersionError`
- Depending on Hibernate directly for basics — ties the war to one runtime

## ✓ TIPS & BEST PRACTICES

- Use the runtime's Maven plugin dev mode for redeploy on save
- Add vendor jars (Hibernate extras) as `provided` when the server has them
- Match `jakartaee-web-api` to TomEE/Web-only servers so Platform APIs fail at compile time

## QUICK SUMMARY

- `provided` — compile against, don't ship
- `jakartaee-api` — one dependency for all specs
- `war` — standard deployable
- `dev` goals — live redeploy

Sheet 04 · Servlet 6.1

# App Structure & Deployment

WEB-INF/ · web.xml · beans.xml · persistence.xml · war vs ear · context root

## DESCRIPTION

A war has a fixed layout: public files at the root, and everything under **WEB-INF/** hidden from browsers. Since annotations replaced most XML, `web.xml` is optional and `beans.xml` can be empty or absent. Deploy by copying the war to the server's deploy folder, using its CLI, or with a Maven plugin.

## NOTES

- `src/main/webapp/` becomes the war root
- `META-INF/persistence.xml` lives under `src/main/resources/`
- An **ear** bundles several wars and EJB jars; rarely needed today
- Context root defaults to the war name

## PROJECT → WAR LAYOUT

```
src/main/java/com/shop/...        → WEB-INF/classes/
src/main/resources/META-INF/
    persistence.xml               → WEB-INF/classes/META-INF/
src/main/webapp/
    index.html                    → /index.html (public)
    WEB-INF/
        web.xml        (optional) → servlet config, security
        beans.xml      (optional) → CDI discovery mode
        lib/                      → third-party jars
```

## CODE EXPLAINED

- → `WEB-INF/` — never served to clients; safe for config
- → `beans.xml` absent → **annotated** discovery (bean-defining annotations only)
- → `web.xml` still needed for some security and error-page setups
- → Hot deploy: replacing the war redeploys it

## SPRING EQUIVALENT

- Boot has no war layout: `src/main/resources/static/` and `application.properties`
- No `beans.xml` — component scan from `@SpringBootApplication` · See Spring 04

## DEPLOY

```
cp target/shop.war $WILDFLY_HOME/standalone/deployments/
$PAYARA/bin/asadmin deploy --contextroot shop target/shop.war
mvn wildfly:deploy                  # via the Maven plugin
```

## PREFER / AVOID

- ⇄ ~~`web.xml`~~ servlet mappings → `@WebServlet`
- ⇄ secrets in ~~`webapp/`~~ → `WEB-INF/` or server config
- ⇄ ear for a single web app → plain war

## DESCRIPTORS

| FILE | PURPOSE | NEEDED? |
|---|---|---|
| `web.xml` | servlets, filters, security, error pages | optional |
| `beans.xml` | CDI `bean-discovery-mode` | optional |
| `persistence.xml` | persistence units, datasource | yes, for JPA |
| `glassfish-web.xml etc.` | vendor-specific settings | per runtime |

## MINIMAL BEANS.XML

| ATTRIBUTE | VALUE |
|---|---|
| `version` | `4.1` |
| `bean-discovery-mode` | `annotated` (default) · `all` · `none` |
| `namespace` | `https://jakarta.ee/xml/ns/jakartaee` |

## ⚠ COMMON MISTAKES

- Putting `persistence.xml` in `webapp/META-INF` — JPA only reads it from the classpath
- Assuming an empty `beans.xml` means `all` — since CDI 4 it means `annotated`
- Two wars with the same context root — the second deploy fails or replaces the first

## ✓ TIPS & BEST PRACTICES

- Set the context root in the vendor descriptor or deploy command, not by renaming the war
- Keep `web.xml` only for things annotations can't express
- Watch the server log on deploy — CDI errors show up there, not at compile time

## QUICK SUMMARY

- `WEB-INF/` — private config and classes
- `beans.xml` — discovery mode
- `persistence.xml` — classpath `META-INF/`
- `deployments/` — drop-in deploy

Sheet 05 | java.beans (Java SE)

# JavaBeans Conventions

no-arg constructor · getX / setX · isX · Serializable · properties vs fields · records

## DESCRIPTION

A **JavaBean** is a plain class that follows naming conventions so tools can work with it by reflection: a public no-arg constructor, private fields, and **properties** exposed through `getX()` / `setX()` methods (`isX()` for booleans). The property name comes from the method, not the field. Faces, EL, JPA and JSON-B all rely on these conventions.

## NOTES

- Defined by the JavaBeans spec in Java SE (`java.beans`), not Jakarta
- `getFirstName()` → property `firstName`; `getURL()` → `URL`
- Records use `name()` accessors — EL 6 and JSON-B 3 understand them

## A TEXTBOOK JAVABEAN

```
public class Customer implements Serializable {
    private String name;                    // backing field
    private boolean active;
    private int loyaltyPoints;

    public Customer() { }                   // required no-arg ctor

    public String getName() { return name; }           // property "name"
    public void setName(String name) { this.name = name; }

    public boolean isActive() { return active; }       // boolean: isX
    public void setActive(boolean active) { this.active = active; }

    public int getLoyaltyPoints() { return loyaltyPoints; }
    // no setter → read-only property
}
```

## CODE EXPLAINED

- → `Serializable` — needed for session-scoped beans and passivation
- → `isActive()` — only for primitive `boolean`; `Boolean` uses `getActive()`
- → A getter alone = **read-only** property; a setter alone = write-only
- → IntelliJ ⌘N › Getter and Setter generates these (IntelliJ 09)

## SPRING EQUIVALENT

- Spring binds form fields and `@ConfigurationProperties` to the same setters
- Spring's `BeanWrapper` = `Introspector` + type conversion · See Spring 15

## PREFER / AVOID

- ⇄ public fields for EL → private field + getter
- ⇄ ~~`getIsActive()`~~ → `isActive()`
- ⇄ mutable bean for DTOs → record (JSON-B, EL and Validation support it)

## NAMING RULES

| METHOD | PROPERTY | TYPE |
|---|---|---|
| `getName()` | name | read |
| `setName(String)` | name | write |
| `isActive()` | active | boolean read |
| `getURL()` | URL | 2 capitals keep case |

## WHERE CONVENTIONS MATTER

| TECHNOLOGY | USES |
|---|---|
| EL / Faces | `#{customer.name}` calls `getName()` |
| JPA | property access mode reads getters |
| JSON-B | serializes getters / public fields |
| Bean Validation | constraints on getters or fields |

## ⚠ COMMON MISTAKES

- No no-arg constructor — frameworks that instantiate by reflection throw `InstantiationException`
- `getIsActive()` — EL looks for `isActive()` / `getActive()` and reports the property missing
- Mismatched types — `getAge()` returns `int` but `setAge(String)` makes `age` read-only to tools

## ✓ TIPS & BEST PRACTICES

- Generate accessors with the IDE; hand-typed names drift
- Use records for immutable data, classic beans for Faces forms that need setters

## QUICK SUMMARY

- `no-arg ctor` — reflection can create it
- `getX / setX` — property accessors
- `isX` — primitive boolean getter
- `record` — accessor `x()` supported by EL/JSON-B

Sheet 06 · java.beans (Java SE)

# Introspection & Property Change Events

Introspector · BeanInfo · PropertyDescriptor · PropertyChangeSupport · bound · constrained

## DESCRIPTION

**Introspection** is how tools discover a bean's properties at runtime: `Introspector.getBeanInfo()` returns a `BeanInfo` with a `PropertyDescriptor` for each getter/setter pair. **Bound properties** fire a `PropertyChangeEvent` when they change; **constrained** properties let listeners veto a change by throwing `PropertyVetoException`.

## NOTES

- `PropertyChangeSupport` does the listener bookkeeping for you
- Events fire only if old and new values differ ( `equals` )
- Swing uses bound properties heavily — see Swing set
- `Introspector.flushCaches()` after hot-reloading classes

## UNDER THE CONTAINER

- Properties come from **method names**, not fields — a field with no getter is invisible
- `BeanInfo` results are cached per class, so introspection is cheap after the first call
- EL resolves `#{user.name}` via `BeanELResolver`, which calls the descriptor's read method

## BOUND PROPERTY + INTROSPECTION

```
public class Order {
    private final PropertyChangeSupport pcs =
        new PropertyChangeSupport(this);
    private String status = "NEW";

    public String getStatus() { return status; }
    public void setStatus(String s) {
        String old = status; status = s;
        pcs.firePropertyChange("status", old, s);  // bound
    }
    public void addPropertyChangeListener(PropertyChangeListener l) {
        pcs.addPropertyChangeListener(l);
    }
}
order.addPropertyChangeListener(e ->
    log.info(e.getPropertyName() + ": " + e.getOldValue()
             + " → " + e.getNewValue()));    // status: NEW → PAID
order.setStatus("PAID");

BeanInfo info = Introspector.getBeanInfo(Order.class, Object.class);
for (PropertyDescriptor pd : info.getPropertyDescriptors())
    System.out.println(pd.getName());        // status
```

## CODE EXPLAINED

- → `firePropertyChange` — skips the event when old equals new
- → `getBeanInfo(cls, Object.class)` — stop class excludes `getClass()`
- → `PropertyDescriptor` — `getReadMethod()`, `getWriteMethod()`, `getPropertyType()`

## SPRING EQUIVALENT

- Spring `BeanUtils.getPropertyDescriptors()` wraps the same API
- Spring app events ( `ApplicationEventPublisher` ) replace bean events for app logic · See Spring 60

## PREFER / AVOID

⇄ hand-rolled listener lists → `PropertyChangeSupport`
⇄ reflection on fields → `Introspector` + descriptors
⇄ bean events for business workflows → CDI events (sheet 14)

## KEY CLASSES

| CLASS | ROLE |
|---|---|
| `Introspector` | builds `BeanInfo` for a class |
| `PropertyDescriptor` | one property: read/write methods |
| `PropertyChangeSupport` | fires bound-property events |
| `VetoableChangeSupport` | fires constrained events |

## PROPERTY KINDS

| KIND | BEHAVIOR |
|---|---|
| `simple` | getter / setter only |
| `bound` | fires `PropertyChangeEvent` after change |
| `constrained` | fires before; listener may throw `PropertyVetoException` |
| `indexed` | `getX(int)` / `setX(int, v)` |

## ⚠ COMMON MISTAKES

- Firing before assigning the field — listeners that read the bean see the old value
- Forgetting the stop class — `class` shows up as a property from `getClass()`
- Using bound properties across threads — listeners run on the caller's thread, not the UI thread

## ✓ TIPS & BEST PRACTICES

- Make `PropertyChangeSupport` field `final` and `transient` if the bean is serialized
- Use `fireIndexedPropertyChange` for list-like properties
- Prefer CDI events in server code; keep bean events for UI models

## QUICK SUMMARY

- `Introspector` — discovers properties
- `PropertyDescriptor` — getter/setter pair
- `bound` — notifies after change
- `constrained` — listeners can veto

# Which Bean Is Which?



## DESCRIPTION

"Bean" means six different things in Jakarta EE. They all start from **JavaBeans** conventions, but differ in **who creates them**, what **scope** they live in, and what services they get. Most modern code needs only two: **CDI beans** for logic and **entities** for data. Use this sheet to decode legacy code and tutorials.

## NOTES

- A **Faces backing bean** is just a `@Named` CDI bean today
- `@ManagedBean` (Faces) was removed in Faces 4.0
- One class can be several kinds — an entity can also be validated
- **Never** inject an entity; load it via `EntityManager`

## BEAN TYPES COMPARED

| TYPE | MARKER | CREATED BY | SCOPE | TRANSACTIONS |
|---|---|---|---|---|
| JavaBean | conventions only | you (`new`) | none | no |
| CDI bean | `@ApplicationScoped` etc. | CDI container | context scopes | `@Transactional` |
| Session bean | `@Stateless` `@Stateful` `@Singleton` | EJB container | pooled / client / one | yes, by default |
| Message-driven | `@MessageDriven` | EJB container | pooled | yes, by default |
| Backing bean | `@Named` + scope | CDI container | view / request / session | `@Transactional` |
| JPA entity | `@Entity` | `EntityManager` | persistence context | via caller |

## CODE EXPLAINED

→ Logic → **CDI bean**; data → **entity**; forms → `@Named` CDI bean

→ Need timers, pooling or MDBs → **EJB** (sheets 08–10)

## SPRING EQUIVALENT

- CDI bean → `@Component` / `@Service`
- EJB `@Stateless` → `@Service` + `@Transactional`
- Entity → same JPA `@Entity` · See Spring 36

## PREFER / AVOID

⇄ ~~`@ManagedBean`~~ → `@Named` + CDI scope

⇄ ~~`@Stateless`~~ just for transactions → CDI bean + `@Transactional`

⇄ entity as REST DTO → record DTO (sheet 23)

## LIFECYCLE CALLBACKS

| CALLBACK | APPLIES TO |
|---|---|
| `@PostConstruct` | CDI, EJB, Faces beans |
| `@PreDestroy` | CDI, EJB, Faces beans |
| `@PrePersist` / `@PostLoad` | JPA entities |

## WHERE TO READ MORE

| TYPE | SHEETS |
|---|---|
| JavaBean · EJB | 05–06 · 08–10 |
| CDI | 11–14 |
| Faces backing bean | 18–19 |
| Entity · Validation | 25–27 · 31 |

## ⚠ COMMON MISTAKES

- Injecting with `@Inject` into a plain `new`'d JavaBean — only container-created beans get injection
- Marking an entity `@ApplicationScoped` — entities aren't CDI beans; scopes don't apply
- Keeping `@ManagedBean` from an old tutorial — removed in Faces 4, the bean is never found

## ✓ TIPS & BEST PRACTICES

- When reading old code, check imports first: `jakarta.ejb`, `jakarta.enterprise`, `jakarta.persistence`
- Default to CDI; reach for EJB only for features CDI lacks
- Keep entities free of service logic so they stay simple JavaBeans

## QUICK SUMMARY

- `CDI bean` — default for logic
- `EJB` — pooling, timers, MDBs
- `@Entity` — managed by `EntityManager`
- `@Named` — EL name for Faces

# Stateless & Singleton Session Beans



## DESCRIPTION

**Session beans** are EJB components the container pools, secures and wraps in transactions. A **`@Stateless` bean holds no per-client state, so any pooled instance can serve any call. A `@Singleton`** has exactly one instance per app, with container-managed locking. Both get a container transaction on every public method by default.

## NOTES

- No-interface view: inject the class itself; `@Local` only for interfaces
- `@Startup` creates a singleton eagerly at deploy
- Singleton default lock is **WRITE** — one caller at a time
- Inject with `@Inject` (CDI) rather than `@EJB`

## UNDER THE CONTAINER

- Stateless instances are **pooled**; never keep caller data in fields
- Every business method runs in a container transaction ( `REQUIRED` ) unless told otherwise
- `this.other()` skips the proxy — no new transaction, no security check

## STATELESS SERVICE + SINGLETON CACHE

```
@Stateless                                  // pooled, transactional
public class OrderService {
    @PersistenceContext EntityManager em;
    public void place(Order o) { em.persist(o); }   // tx per call
}

@Singleton @Startup                         // one instance, eager
@ConcurrencyManagement(ConcurrencyManagementType.CONTAINER)
public class RateCache {
    private Map<String, BigDecimal> rates = new HashMap<>();

    @PostConstruct void load() { rates = fetchRates(); }

    @Lock(LockType.READ)                    // many readers at once
    public BigDecimal rate(String ccy) { return rates.get(ccy); }

    @Lock(LockType.WRITE)                   // exclusive
    public void refresh() { rates = fetchRates(); }
}
```

## CODE EXPLAINED

- → `@Stateless` — container picks a free instance per call
- → `@Startup` — runs `@PostConstruct` during deployment
- → `@Lock(READ)` — concurrent callers; `WRITE` blocks everyone
- → `BEAN` concurrency → you use `synchronized` yourself

## SPRING EQUIVALENT

- `@Stateless` → `@Service` + `@Transactional` (Spring beans aren't pooled)
- `@Singleton` `@Startup` → `@Component` + `ApplicationReadyEvent` · See Spring 42

## PREFER / AVOID

- ⇄ ~~`@Stateless`~~ only for transactions → `@ApplicationScoped` + `@Transactional`
- ⇄ ~~`@EJB`~~ field injection → `@Inject`
- ⇄ WRITE lock on read methods → `@Lock(READ)`

## SESSION BEAN KINDS

| TYPE | INSTANCES | STATE |
|---|---|---|
| `@Stateless` | pool | none between calls |
| `@Singleton` | one per app | shared, locked |
| `@Stateful` | one per client | conversational (sheet 09) |

## VIEWS & INJECTION

| FORM | USE |
|---|---|
| `no-interface` | inject the class; default |
| `@Local` | local interface view |
| `@Remote` | remote calls — avoid |
| `@Inject / @EJB` | prefer `@Inject` |

## ⚠ COMMON MISTAKES

- Storing request data in a stateless field — the next caller may get the same instance and see it
- Long work under a WRITE lock — every other call to the singleton waits or times out
- Calling a `REQUIRES_NEW` method on `this` — the annotation is ignored

## ✓ TIPS & BEST PRACTICES

- Use `@AccessTimeout` to fail fast instead of piling up on a lock
- Inject `SessionContext` and call `getBusinessObject()` to self-call through the proxy
- For new code, try a CDI bean first (sheet 10)

## QUICK SUMMARY

- `@Stateless` — pooled, no state
- `@Singleton` — one, container-locked
- `@Lock(READ/WRITE)` — concurrency
- `@Startup` — eager init

# Stateful Beans, Lifecycle & Timers



## DESCRIPTION

A **`@Stateful` bean keeps state for one client across calls (a shopping cart), and the container may** passivate **it to disk when idle. EJBs also give you** timers** (`@Schedule` for cron-style, `TimerService` for programmatic) and **`@Asynchronous`** methods that return a `Future`. Timers are the main reason EJB survives in new code.

## NOTES

- `@Remove` ends a stateful bean; otherwise it times out
- Passivated beans must be `Serializable` (fields too)
- `@Schedule(persistent=false)` skips the timer database
- EE 11 Concurrency `@Asynchronous` is the CDI alternative

## CART, NIGHTLY JOB AND ASYNC CALL

```
@Stateful @StatefulTimeout(value = 30, unit = MINUTES)
public class Cart implements Serializable {
    private final List<Item> items = new ArrayList<>();
    public void add(Item i) { items.add(i); }
    @Remove public Order checkout() { return Order.of(items); }
}

@Singleton
public class Jobs {
    @Schedule(hour = "2", minute = "0", persistent = false)
    void nightlyCleanup() { /* 02:00 every day */ }

    @Resource TimerService timers;
    public void remindIn(Duration d) {
        timers.createSingleActionTimer(d.toMillis(), new TimerConfig());
    }
    @Timeout void onTimer(Timer t) { /* fires once */ }

    @Asynchronous
    public Future<Report> build() { return new AsyncResult<>(make()); }
}
```

## CODE EXPLAINED

- → `@StatefulTimeout` — idle time before the container removes it
- → `@Remove` — call it to end the conversation and free memory
- → `@Schedule` — cron fields: `second minute hour dayOfMonth month dayOfWeek year`
- → `@Timeout` — callback for programmatic timers

## SPRING EQUIVALENT

- `@Schedule` → `@Scheduled(cron=…)` (Spring 61)
- `@Asynchronous` → `@Async` + `CompletableFuture` (Spring 62)
- `@Stateful` → `@SessionScope` bean

## PREFER / AVOID

- ⇄ ~~`AsyncResult` + `Future`~~ → `CompletableFuture` (EJB 3.2+)
- ⇄ persistent timers by default → `persistent = false` unless they must survive restarts
- ⇄ stateful EJB for web carts → `@SessionScoped` CDI bean

## LIFECYCLE CALLBACKS

| CALLBACK | WHEN |
|---|---|
| `@PostConstruct` | after injection |
| `@PreDestroy` | before removal |
| `@PrePassivate` | before writing to disk |
| `@PostActivate` | after reading back |

## @SCHEDULE EXAMPLES

| ATTRIBUTES | FIRES |
|---|---|
| `minute="*/15", hour="*"` | every 15 min |
| `hour="2"` | 02:00 daily |
| `dayOfWeek="Mon-Fri", hour="9"` | weekdays 09:00 |
| `dayOfMonth="Last"` | last day of month, midnight |

## ⚠ COMMON MISTAKES

- Forgetting `hour = "*"` — `@Schedule` defaults hour to 0, so `minute="*/5"` only fires during midnight
- Non-serializable fields in a stateful bean — passivation fails and the bean is discarded
- Never calling `@Remove` — stateful beans pile up until timeout

## ✓ TIPS & BEST PRACTICES

- Put `@Schedule` methods on a `@Singleton` so two nodes don't double-run without clustering
- Use `timers.getTimers()` to list and cancel active timers
- Prefer `ManagedScheduledExecutorService` (sheet 33) when you don't need persistence

## QUICK SUMMARY

- `@Stateful` — one per client
- `@Remove` — end conversation
- `@Schedule` — cron timer
- `@Asynchronous` — returns a `Future`

# Enterprise Beans vs CDI Beans



## DESCRIPTION

Since Java EE 7, most EJB services are available to plain **CDI beans**: `@Transactional`, `@RolesAllowed`, interceptors and managed executors. The platform's direction is **CDI-first**; EE 12 continues moving EJB features into CDI-based specs. Keep EJB where it still adds something unique, and write new services as CDI beans.

## NOTES

- Core and Web Profiles include only **EJB Lite** or none
- EJB Lite: session beans, no MDBs, no remote, no timers in Core
- A class can't be both `@Stateless` and `@ApplicationScoped`
- Converting is usually just swapping the annotation

## FEATURE BY FEATURE

| NEED | EJB | CDI (+ OTHER SPECS) |
|---|---|---|
| transactions | default on every method | `@Transactional` |
| security | `@RolesAllowed` | `@RolesAllowed` (Security) |
| async | `@Asynchronous` | `@Asynchronous` (Concurrency 3) |
| scheduling | `@Schedule`, persistent timers | `@Schedule` (Concurrency 3.1), no persistence |
| pooling | built-in instance pool | none — beans are shared |
| locking | `@Lock` on singletons | `synchronized` / `j.u.c` locks |
| messaging | `@MessageDriven` | no equivalent yet |
| remote calls | `@Remote` | use REST instead |

## CODE EXPLAINED

→ Pick **CDI** by default; it's lighter and works in every profile

→ Keep **EJB** for MDBs, persistent timers and pooled throttling

→ Mixed apps are normal — CDI can inject EJBs and vice versa

## SPRING EQUIVALENT

- Spring never had EJBs; every Spring bean is CDI-style
- Spring `@Transactional` = CDI `@Transactional` semantics · See Spring 42

## PREFER / AVOID

⇄ new ~~`@Stateless`~~ service → `@ApplicationScoped` + `@Transactional`

⇄ EJB ~~`@Asynchronous`~~ → Concurrency `@Asynchronous`

⇄ ~~`@Remote`~~ EJB → REST resource

## MIGRATION MAP

| EJB | CDI REPLACEMENT |
|---|---|
| `@Stateless` | @ApplicationScoped + @Transactional |
| `@Singleton + @Lock` | @ApplicationScoped + locks |
| `@Stateful` | @SessionScoped / @ConversationScoped |
| `@TransactionAttribute` | @Transactional(TxType…) |

## KEEP EJB WHEN

| FEATURE | WHY |
|---|---|
| `@MessageDriven` | JMS consumers (sheet 34) |
| persistent timers | survive restarts |
| instance pool | caps concurrency |
| legacy remote clients | can't change callers yet |

## ⚠ COMMON MISTAKES

- Assuming CDI `@ApplicationScoped` is pooled — it's one shared instance, so fields need thread safety
- Swapping to CDI but forgetting `@Transactional` — writes now run without a transaction
- Converting MDBs to CDI — there's no CDI message listener; keep the MDB

## ✓ TIPS & BEST PRACTICES

- Migrate one bean at a time and run integration tests between steps
- Grep for `@TransactionAttribute` to find tx settings that must move to `@Transactional`
- Watch EE 12 milestones for further EJB-to-CDI moves

## QUICK SUMMARY

- `CDI-first` — default for new beans
- `@Transactional` — CDI transactions
- `EJB` — MDBs, timers, pools
- `EJB Lite` — Web Profile subset

Sheet 11 · CDI 4.1

# CDI Basics & Injection

@Inject · @Named · bean-defining annotations · discovery modes · CDI Lite · Instance<T>

## DESCRIPTION

**CDI** (Contexts and Dependency Injection) is the Jakarta EE component model. The container discovers beans, creates them in the right **scope**, and wires dependencies by **type** with `@Inject`. Constructor injection is the preferred form. **CDI Lite** (Core Profile) works at build time; **CDI Full** adds decorators, conversations and portable extensions.

## NOTES

- A class becomes a bean via a **bean-defining annotation** (any scope, `@Dependent`, stereotype)
- Default discovery is `annotated` — plain classes aren't beans
- `@Named` gives an EL name: `#{orderService}`
- Quarkus and Helidon implement CDI Lite at build time

## UNDER THE CONTAINER

- Normal-scoped beans are injected as a **client proxy**, not the real instance
- `annotated` mode scans only classes with bean-defining annotations; `all` makes every class a bean
- CDI Lite resolves injection at build time, so errors appear when you compile

## THREE WAYS TO INJECT

```
@ApplicationScoped
public class CheckoutService {
    private final PaymentGateway gateway;
    private final Event<OrderPlaced> events;

    @Inject                                  // constructor (preferred)
    public CheckoutService(PaymentGateway g, Event<OrderPlaced> e) {
        this.gateway = g; this.events = e;
    }
    protected CheckoutService() { this(null, null); } // proxy needs it

    @Inject Clock clock;                     // field
    @Inject void setAudit(AuditLog a) { }    // initializer method

    @Inject Instance<Notifier> notifiers;    // dynamic / optional
    void notifyAll(Order o) {
        notifiers.forEach(n -> n.send(o));   // every Notifier bean
        if (notifiers.isUnsatisfied()) log.warn("no notifier");
    }
}
```

## CODE EXPLAINED

- → Constructor injection → `final` fields, easy unit tests
- → Normal-scoped beans need a non-private no-arg ctor for the proxy
- → `Instance<T>` — lazy, optional or multiple beans; `get()`, `select()`
- → Ambiguous or missing beans fail **at deploy**, not at runtime

## SPRING EQUIVALENT

- `@Inject` → `@Autowired` or plain constructor injection
- `Instance<T>` → `ObjectProvider<T>`
- `@Named` → bean name / `@Qualifier` · See Spring 08

## PREFER / AVOID

- ⇄ field injection everywhere → constructor injection
- ⇄ ~~`CDI.current().select()`~~ lookups → `@Inject`
- ⇄ ~~`bean-discovery-mode="all"`~~ → `annotated` + explicit scopes

## BEAN-DEFINING ANNOTATIONS

| ANNOTATION | MAKES A BEAN |
|---|---|
| `@ApplicationScoped` | yes |
| `@RequestScoped / @SessionScoped` | yes |
| `@Dependent` | yes (explicit) |
| `@Stereotype (e.g. @Model)` | yes |
| `@Named alone` | no |

## CDI LITE VS FULL

| FEATURE | LITE | FULL |
|---|---|---|
| `@Inject, scopes, producers` | ✔ | ✔ |
| `interceptors, events` | ✔ | ✔ |
| `decorators` | ✖ | ✔ |
| `@ConversationScoped` | ✖ | ✔ |
| `portable extensions` | `build-time` | runtime |

## ⚠ COMMON MISTAKES

- `@Named` without a scope — under `annotated` discovery it isn't a bean at all
- Private no-arg constructor on an `@ApplicationScoped` bean — the proxy can't subclass it
- Injecting into a static field — CDI ignores statics

## ✓ TIPS & BEST PRACTICES

- Read deploy logs for `WELD-001408` (unsatisfied) and `WELD-001409` (ambiguous)
- Use `@Model` stereotype (= `@Named @RequestScoped`) for page models
- Keep `beans.xml` out unless you need `all` or alternatives

## QUICK SUMMARY

- `@Inject` — type-safe injection
- `annotated` — default discovery
- `Instance<T>` — optional / many
- `CDI Lite` — build-time subset

# Scopes & Lifecycle



## DESCRIPTION

A **scope** decides how long a bean instance lives and who shares it. **Normal scopes** (request, session, application, conversation) are reached through a client proxy, so a long-lived bean can safely hold a short-lived one. **@Dependent** is a pseudo-scope: a new instance per injection point that lives and dies with its owner.

## NOTES

- `@Dependent` is the default when no scope is given (with `all` discovery)
- Session- and conversation-scoped beans must be `Serializable`
- Faces adds `@ViewScoped` and `@FlowScoped`

## UNDER THE CONTAINER

- Each scope has a context that stores its instances; the proxy looks up the current one per call
- A `@RequestScoped` bean inside an `@ApplicationScoped` one works: the proxy finds the current request

## ONE BEAN PER SCOPE

```
@RequestScoped                              // one per HTTP request
public class RequestInfo { String traceId = UUID.randomUUID().toString(); }

@SessionScoped                              // one per user session
public class UserPrefs implements Serializable { Locale locale; }

@ApplicationScoped                          // one for the whole app
public class Catalog {
    @Inject RequestInfo req;                // proxy → current request
    @PostConstruct void warm() { loadAll(); }    // once
}

@ConversationScoped                         // spans several requests
public class Wizard implements Serializable {
    @Inject Conversation conv;
    void start() { conv.begin(); }          // until conv.end()
    void finish() { conv.end(); }
}
```

## CODE EXPLAINED

- → `RequestInfo` gets a new `traceId` every request, even inside `Catalog`
- → `@PostConstruct` — after injection; `@PreDestroy` — before discard
- → `Conversation.begin()` / `end()` — `?cid=` carries it between requests

## SPRING EQUIVALENT

- `@RequestScoped` → `@RequestScope`, `@SessionScoped` → `@SessionScope`
- `@ApplicationScoped` → default singleton
- `@Dependent` → prototype · See Spring 11

## PREFER / AVOID

- ⇄ mutable app-scoped state → narrower scope
- ⇄ storing user data in static fields → `@SessionScoped`
- ⇄ conversation for one page → `@ViewScoped`

## SCOPES

| SCOPE | LIVES FOR | PROXY? |
|---|---|---|
| `@RequestScoped` | one request | yes |
| `@SessionScoped` | HTTP session | yes |
| `@ApplicationScoped` | app lifetime | yes |
| `@ConversationScoped` | begin() → end() | yes |
| `@Dependent` | its owner | no |

## LIFECYCLE ORDER

| STEP | WHAT HAPPENS |
|---|---|
| 1. construct | constructor (with injection) |
| 2. inject | fields, initializer methods |
| 3. @PostConstruct | init logic |
| 4. @PreDestroy | context ends |

## ⚠ COMMON MISTAKES

- `final` methods on a scoped bean — the proxy can't intercept them and you read its empty fields
- Expecting `@PostConstruct` at startup — `@ApplicationScoped` is lazy; use `@Observes Startup`
- Non-serializable `@SessionScoped` bean — session replication and passivation fail

## ✓ TIPS & BEST PRACTICES

- Pick the narrowest scope that works; it avoids thread-safety bugs

## QUICK SUMMARY

- `normal scope` — shared through a proxy
- `@Dependent` — per injection point
- `@PostConstruct` — after injection
- `@ConversationScoped` — multi-request

Sheet 13 · CDI 4.1

# Qualifiers, Producers & Alternatives

@Qualifier · @Default · @Produces · @Disposes · InjectionPoint · @Alternative · @Priority

## DESCRIPTION

When two beans share a type, a **qualifier** annotation picks one. **Producer methods** turn anything (third-party objects, config values, per-injection values) into injectable beans, with optional **disposers** for cleanup. **Alternatives** swap an implementation per deployment, enabled globally with `@Priority`.

## NOTES

- Every bean has `@Any`; beans without a qualifier also get `@Default`
- Producer scope comes from an annotation on the producer method
- `InjectionPoint` works only in `@Dependent` producers
- Built-in `@Named` is a qualifier too, but string-based

## QUALIFIER, PRODUCER, ALTERNATIVE

```
@Qualifier @Retention(RUNTIME) @Target({FIELD, PARAMETER, TYPE})
public @interface Express { }

@Express @ApplicationScoped
public class ExpressShipping implements Shipping { }
@ApplicationScoped
public class GroundShipping implements Shipping { }     // @Default

@Inject @Express Shipping fast;            // ExpressShipping
@Inject Shipping normal;                   // GroundShipping

public class Resources {
    @Produces @ApplicationScoped
    HttpClient http() { return HttpClient.newHttpClient(); }
    void close(@Disposes HttpClient c) { c.close(); }

    @Produces Logger log(InjectionPoint ip) {          // per class
        return Logger.getLogger(
            ip.getMember().getDeclaringClass().getName());
    }
}

@Alternative @Priority(1) @ApplicationScoped
public class FakeShipping implements Shipping { }      // wins when present
```

## CODE EXPLAINED

- → Custom qualifier = annotation with `@Qualifier` + `RUNTIME` retention
- → `@Produces` — method return value becomes a bean
- → `@Disposes` — called when the produced instance's scope ends
- → `@Priority` on an `@Alternative` enables it app-wide

## SPRING EQUIVALENT

- Qualifier → `@Qualifier` / `@Primary`
- `@Produces` → `@Bean` method
- `@Alternative` → `@Profile` or `@ConditionalOnProperty` · See Spring 10

## PREFER / AVOID

- ⇄ ~~`@Named("express")`~~ strings → typed qualifier annotation
- ⇄ ~~`new`~~ third-party objects in many classes → one producer
- ⇄ alternatives in ~~`beans.xml`~~ → `@Priority`

## RESOLUTION ERRORS

| ERROR | CAUSE → FIX |
|---|---|
| `Unsatisfied dependency` | no bean matches → add scope/producer |
| `Ambiguous dependency` | 2+ match → add qualifier |
| `Unproxyable type` | final class/method → remove `final` |

## BUILT-IN QUALIFIERS

| QUALIFIER | MEANING |
|---|---|
| `@Default` | no other qualifier given |
| `@Any` | every bean; use with `Instance` |
| `@Named` | EL name; string qualifier |
| `@Intercepted` | bean being intercepted |

## ⚠ COMMON MISTAKES

- Forgetting `RUNTIME` retention — the qualifier is invisible and you get an ambiguity error
- Producer method and a bean class for the same type — ambiguous; qualify one
- Using `InjectionPoint` in an `@ApplicationScoped` producer — not allowed; one instance can't know every caller

## ✓ TIPS & BEST PRACTICES

- Group producers in a `Resources` class so third-party wiring lives in one place
- Use `Instance<Shipping>` with `@Any` to iterate every implementation
- Put test alternatives in `src/test` so they're only present in test deployments

## QUICK SUMMARY

- `@Qualifier` — choose between same-type beans
- `@Produces` — make a bean from a method
- `@Disposes` — cleanup
- `@Alternative + @Priority` — swap impl

Sheet 14 · CDI 4.1 · Interceptors 2.2

# Events, Interceptors & Decorators

Event<T>.fire() · @Observes · @ObservesAsync · @InterceptorBinding · @AroundInvoke · @Decorator

## DESCRIPTION

**Events** decouple beans: a producer fires an object, every matching observer receives it. **Interceptors** wrap method calls with cross-cutting logic (timing, logging, auditing) bound by an annotation. **Decorators** wrap a specific interface to add business behavior. All three run through the same container proxy.

## NOTES

- Sync observers run in the firing thread and transaction
- `@Observes(during = AFTER_SUCCESS)` waits for commit
- Decorators are CDI Full only

## UNDER THE CONTAINER

- Interceptors and decorators are applied through the bean's proxy
- Self-invocation ( `this.save()` ) bypasses them — call through an injected reference
- Lower `@Priority` value runs first (outermost) for interceptors

## EVENT + INTERCEPTOR

```
@Inject Event<OrderPlaced> placed;
void checkout(Order o) { placed.fire(new OrderPlaced(o.id())); }

void email(@Observes OrderPlaced e) { mailer.confirm(e.id()); }
void stats(@ObservesAsync OrderPlaced e) { metrics.inc(); } // fireAsync

@InterceptorBinding @Retention(RUNTIME) @Target({TYPE, METHOD})
public @interface Timed { }

@Timed @Interceptor @Priority(Interceptor.Priority.APPLICATION)
public class TimedInterceptor {
    @AroundInvoke
    Object time(InvocationContext ctx) throws Exception {
        long t0 = System.nanoTime();
        try { return ctx.proceed(); }                  // call the method
        finally { log.info(ctx.getMethod().getName() + " "
                  + (System.nanoTime() - t0) / 1_000_000 + "ms"); }
    }
}
@Timed @ApplicationScoped public class ReportService { }   // all methods
```

## CODE EXPLAINED

- → `fire()` — sync observers only; `fireAsync()` — async observers, returns `CompletionStage`
- → `@InterceptorBinding` — the annotation that links bean → interceptor
- → `ctx.proceed()` — calls the next interceptor or the target
- → `Interceptor.Priority.APPLICATION` = 2000

## SPRING EQUIVALENT

- `Event<T>` → `ApplicationEventPublisher`, `@Observes` → `@EventListener` (Spring 60)
- Interceptor → `@Aspect` + `@Around` (Spring 19)
- `AFTER_SUCCESS` → `@TransactionalEventListener`

## PREFER / AVOID

- ⇄ direct calls to five services → one event, five observers
- ⇄ logging boilerplate in every method → `@Timed` interceptor
- ⇄ side effects before commit → `@Observes(during = AFTER_SUCCESS)`

## TRANSACTION PHASES

| DURING = | RUNS |
|---|---|
| IN_PROGRESS | immediately (default) |
| BEFORE_COMPLETION | just before commit |
| AFTER_COMPLETION | after commit or rollback |
| AFTER_SUCCESS | only after commit |
| AFTER_FAILURE | only after rollback |

## INTERCEPTOR KINDS

| ANNOTATION | WRAPS |
|---|---|
| @AroundInvoke | business methods |
| @AroundConstruct | constructor |
| @AroundTimeout | EJB timer callbacks |

## ⚠ COMMON MISTAKES

- Expecting `fireAsync()` to reach `@Observes` methods — only `@ObservesAsync` observers get async events
- Interceptor without `@Priority` — never enabled unless listed in `beans.xml`
- Throwing from a sync observer — the exception propagates and can roll back the firing transaction

## ✓ TIPS & BEST PRACTICES

- Fire immutable records as event payloads
- Use qualifiers on events ( `Event<@Express OrderPlaced>` ) to target observers
- Put binding annotations on the class to cover every method

## QUICK SUMMARY

- `Event.fire()` — sync broadcast
- `@ObservesAsync` — async observer
- `@AroundInvoke` — wrap calls
- `@Decorator` — typed wrapper (Full)

# Servlets



## DESCRIPTION

A **servlet** is the lowest layer of the Jakarta web tier: a class that receives an `HttpServletRequest` and writes an `HttpServletResponse`. REST, Faces and Spring MVC all run on top of servlets. You rarely write raw servlets now, but knowing them explains filters, sessions and threading everywhere above.

## NOTES

- `@WebServlet` replaces `web.xml` mappings
- Servlet 6.1 (EE 11) adds `sendRedirect` with status and HTTP/2 tweaks
- One instance, many threads — no per-request fields

## UNDER THE CONTAINER

- One servlet instance serves many threads at once, so instance fields are shared
- The filter chain runs before the servlet's `service()` method
- Async servlets hand the request off and free the container thread

## A GET/POST SERVLET

```
@WebServlet(urlPatterns = "/orders/*", asyncSupported = true)
public class OrderServlet extends HttpServlet {
    @Inject OrderService orders;                // CDI works here

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {
        String id = req.getPathInfo().substring(1);    // /orders/42 → 42
        resp.setContentType("application/json");
        resp.getWriter().write(orders.json(id));
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws IOException, ServletException {
        String sku = req.getParameter("sku");    // form or query param
        orders.create(sku);
        resp.sendRedirect(req.getContextPath() + "/orders");  // 302
    }
}
```

## CODE EXPLAINED

- → `urlPatterns` — `/x` exact, `/x/*` prefix, `*.do` extension
- → `getPathInfo()` — part after the servlet path
- → `getParameter` — query string or form body, first value
- → `sendRedirect` — 302 to a new URL; `forward` stays server-side

## SPRING EQUIVALENT

- Spring MVC's `DispatcherServlet` is one servlet that routes to controllers
- `@WebServlet` → `@RestController` method · See Spring 20

## PREFER / AVOID

- ⇄ ~~`web.xml` `<servlet-mapping>`~~ → `@WebServlet`
- ⇄ raw servlets for JSON APIs → Jakarta REST (sheet 21)
- ⇄ string concatenation into HTML → templates (Faces/Pages)

## REQUEST API

| METHOD | RETURNS |
|---|---|
| `getMethod()` | GET, POST… |
| `getRequestURI()` | `/shop/orders/42` |
| `getContextPath()` | `/shop` |
| `getHeader("Accept")` | header value |
| `getSession()` | `HttpSession` (creates) |

## RESPONSE & DISPATCH

| METHOD | EFFECT |
|---|---|
| `setStatus(404)` | status code |
| `sendError(404, msg)` | error page |
| `setHeader(k, v)` | response header |
| `getRequestDispatcher(p).forward()` | server-side forward |
| `startAsync()` | async processing |

## ⚠ COMMON MISTAKES

- Instance fields for request data — two concurrent requests overwrite each other
- Writing to the response after `sendRedirect` — throws `IllegalStateException`
- Hard-coding `/shop` — use `getContextPath()` so it works under any context root

## ✓ TIPS & BEST PRACTICES

- Set the content type and encoding before calling `getWriter()`
- Use `asyncSupported = true` on every filter in the chain too
- Test with `curl -i` to see status and headers

## QUICK SUMMARY

- `HttpServlet` — base class
- `@WebServlet` — URL mapping
- `doGet / doPost` — handlers
- `forward / sendRedirect` — dispatch

# Filters, Listeners & Sessions



## DESCRIPTION

**Filters** wrap every request that matches their pattern, before and after the target runs: logging, auth checks, headers, compression. **Listeners** react to lifecycle events (app start, session created). **Sessions** keep per-user state on the server, identified by a `JSESSIONID` cookie.

## NOTES

- Filter order: `web.xml` order; annotation order is unspecified
- Session timeout default is server-specific (often 30 min)
- `getSession(false)` returns null instead of creating one
- Mark session cookies `HttpOnly` and `Secure`

## FILTER, LISTENER, SESSION

```
@WebFilter("/*")
public class TimingFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res,
            FilterChain chain) throws IOException, ServletException {
        long t0 = System.nanoTime();
        chain.doFilter(req, res);                    // run the rest
        log.info("took " + (System.nanoTime() - t0) / 1e6 + " ms");
    }
}

@WebListener
public class Startup implements ServletContextListener {
    public void contextInitialized(ServletContextEvent e) {
        log.info("app started");
    }
}

HttpSession s = req.getSession();                    // create if absent
s.setAttribute("cartId", 42);
Integer cart = (Integer) s.getAttribute("cartId");   // 42
s.setMaxInactiveInterval(15 * 60);                   // seconds
s.invalidate();                                      // logout
```

## CODE EXPLAINED

- → `chain.doFilter` — skip it to block the request (e.g. 401)
- → `ServletContextListener` — app start/stop hooks
- → `setMaxInactiveInterval` — per-session idle timeout in seconds
- → `invalidate()` — drops all attributes and the id

## SPRING EQUIVALENT

- Filter → `OncePerRequestFilter` / `HandlerInterceptor` (Spring 27)
- Session → `@SessionScope` bean or Spring Session
- Listener → `ApplicationReadyEvent`

## PREFER / AVOID

- ⇄ session attributes for user data → `@SessionScoped` CDI bean
- ⇄ auth checks in every servlet → one filter or Jakarta Security
- ⇄ ~~`HttpSessionListener`~~ counters → metrics (MicroProfile)

## LISTENER TYPES

| INTERFACE | EVENT |
|---|---|
| `ServletContextListener` | app start / stop |
| `HttpSessionListener` | session created / destroyed |
| `ServletRequestListener` | request start / end |
| `HttpSessionAttributeListener` | attribute added / removed |

## COOKIE & SESSION CONFIG

| SETTING | WHERE |
|---|---|
| `timeout` | `<session-timeout>` in `web.xml` |
| `HttpOnly / Secure` | `<cookie-config>` |
| `SameSite` | `cookie-config` attribute (Servlet 6) |
| `tracking` | `<tracking-mode>COOKIE` |

## ⚠ COMMON MISTAKES

- Forgetting `chain.doFilter` — the request silently stops with an empty 200
- Storing large objects in the session — memory per user, and replication cost in clusters
- Using URL rewriting ( `;jsessionid=` ) — leaks session ids in logs and referrers

## ✓ TIPS & BEST PRACTICES

- Cast to `HttpServletRequest` inside filters to read headers and paths
- Rotate the session id after login: `req.changeSessionId()`
- Keep filters thin; heavy logic belongs in services

## QUICK SUMMARY

- `@WebFilter` — wrap requests
- `FilterChain.doFilter` — continue
- `@WebListener` — lifecycle hooks
- `HttpSession` — per-user state

# Pages & Expression Language

Jakarta Pages (JSP) · ${} · #{} · implicit objects · EL lambdas · JavaBean properties

## DESCRIPTION

**Jakarta Expression Language (EL)** is the small expression syntax shared by Pages (JSP) and Faces. It reads JavaBean properties, calls methods and evaluates operators. **${} evaluates immediately (Pages); #{}** is deferred and can also write back to a property (Faces forms). This sheet is a primer; the JSP set covers Pages in depth.

## NOTES

- `#{user.name}` calls `getName()` / `setName()` (sheet 05)
- EL 6 (EE 11) resolves record components and `Optional`
- New UI work usually picks Faces or a JS front end
- See the JSP set for tags, JSTL and directives

## EL IN A PAGE

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<p>Hello, ${user.name}</p>                <!-- getName() -->
<p>Items: ${cart.items.size()}</p>        <!-- method call -->
<p>${empty cart.items ? 'Empty' : 'Has items'}</p>
<p>Total: ${cart.total gt 100 ? 'Free shipping' : ''}</p>
<p>${param.q} · ${header['User-Agent']}</p>
<c:forEach items="${cart.items}" var="i">
  <li>${i.sku} × ${i.qty}</li>
</c:forEach>
```

## CODE EXPLAINED

- → `${}` — evaluate now, read-only
- → `#{}` — deferred; Faces evaluates it in its lifecycle
- → `empty` — true for null, "", empty collection
- → `jakarta.tags.core` — JSTL 3 URI (was `java.sun.com/jsp/jstl/core`)

## SPRING EQUIVALENT

- Thymeleaf `${...}` / `*{...}` plays the same role (Spring 28)
- Spring EL (SpEL) `#{...}` is a different language with similar syntax

## EL LAMBDAS & COLLECTIONS (EL 3+)

```
${[1, 2, 3].stream().map(x -> x * 2).toList()}      // [2, 4, 6]
${orders.stream().filter(o -> o.paid).count()}      // 3
${x = 5; x + 1}                                     // 6
```

## PREFER / AVOID

- ⇄ Java scriptlets `<% %>` → EL + JSTL tags
- ⇄ old JSTL URIs → `jakarta.tags.*`
- ⇄ logic in EL → a bean method

## IMPLICIT OBJECTS

| NAME | HOLDS |
|---|---|
| `param / paramValues` | request parameters |
| `header` | request headers |
| `sessionScope` | session attributes |
| `cookie` | cookies by name |
| `pageContext` | request, response, servlet context |

## OPERATORS

| OP | ALT | EXAMPLE |
|---|---|---|
| `==` | eq | `${a eq b}` |
| `&&` | and | `${a and b}` |
| `<` | lt | `${x lt 3}` |
| `?:` | | `${ok ? 'y' : 'n'}` |
| `+=` | | string concat (EL 3) |

## ⚠ COMMON MISTAKES

- Using `${}` in a Faces input — it's read-only, so the value never posts back; use `#{}`
- Expecting EL to throw on a missing property — it renders empty for null paths, hiding typos
- Old JSTL URIs on EE 10+ — tags silently render as text

## ✓ TIPS & BEST PRACTICES

- Escape output with `<c:out value="${x}"/>` or `fn:escapeXml` in Pages
- Keep EL to property reads; push logic into beans
- Use `${not empty list}` instead of size checks

## QUICK SUMMARY

- `${}` — immediate
- `#{}` — deferred, writable
- `empty` — null/blank check
- `jakarta.tags.core` — JSTL 3

# Faces Basics



## DESCRIPTION

**Jakarta Faces** is a server-side component framework. Pages are **Facelets** XHTML files built from components (`h:inputText`, `h:dataTable`); each component binds to a **backing bean** property through `#{}` EL. On submit, Faces converts, validates and updates the bean, then calls your action method.

## NOTES

- Backing bean = `@Named` CDI bean with a scope ( `@ViewScoped` , `@RequestScoped` )
- Faces 4 removed `@ManagedBean` ; CDI is required

## UNDER THE CONTAINER

- Six phases: restore view → apply values → validate → update model → invoke app → render
- Validation failure skips update + invoke, then re-renders with messages
- View state is saved client- or server-side between requests ( `STATE_SAVING_METHOD` )

## SIGNUP.XHTML

```
<html xmlns:h="jakarta.faces.html" xmlns:f="jakarta.faces.core">
<h:body>
  <h:form>
    <h:inputText value="#{signup.email}" required="true"/>
    <h:inputSecret value="#{signup.password}"/>
    <h:commandButton value="Sign up" action="#{signup.save}"/>
    <h:messages/>
  </h:form>
</h:body></html>
```

## BACKING BEAN

```
@Named @RequestScoped
public class Signup {
    private String email, password;     // + getters / setters
    @Inject UserService users;
    public String save() {
        users.register(email, password);
        return "welcome?faces-redirect=true";   // navigate
    }
}
```

## CODE EXPLAINED

→ `jakarta.faces.html` — Faces 4 namespace URNs (old `xmlns.jcp.org` still work)

→ `value="#{signup.email}"` — reads and writes the property

→ `action` — method returning the next view id, or `null` to stay

## SPRING EQUIVALENT

- Faces has no direct Spring twin; closest is Spring MVC + Thymeleaf forms (Spring 29)
- JoinFaces runs Faces inside Spring Boot

## PREFER / AVOID

⇄ ~~`@ManagedBean`~~ → `@Named` + CDI scope

⇄ ~~`@SessionScoped`~~ for form state → `@ViewScoped`

⇄ ~~`FacesContext.getCurrentInstance()`~~ everywhere → inject `FacesContext`

## COMMON COMPONENTS

| TAG | RENDERS |
|---|---|
| `h:inputText` | `<input type=text>` |
| `h:selectOneMenu` | `<select>` |
| `h:dataTable` | `<table>` from a list |
| `h:commandButton` | submit button |
| `h:link / h:button` | GET navigation |

## USEFUL SCOPES

| SCOPE | USE FOR |
|---|---|
| `@RequestScoped` | simple forms |
| `@ViewScoped` | AJAX on one page |
| `@FlowScoped` | multi-page wizard |
| `@SessionScoped` | logged-in user |

## ⚠ COMMON MISTAKES

- Missing setter — Faces reports "target unreachable" or "property not writable" on submit
- `@ViewScoped` from the wrong package — use `jakarta.faces.view.ViewScoped`
- Putting `h:commandButton` outside `h:form` — clicks do nothing

## ✓ TIPS & BEST PRACTICES

- Set `jakarta.faces.PROJECT_STAGE=Development` for detailed error pages
- Use `?faces-redirect=true` after POST to avoid resubmits
- PrimeFaces or OmniFaces add richer components and utilities

## QUICK SUMMARY

- `Facelets` — XHTML views
- `@Named` bean — backing model
- `#{bean.prop}` — two-way binding
- `action` — returns next view

# Faces Navigation, Validation & AJAX



## DESCRIPTION

Faces navigates by **view id**: an action returning `"orders"` renders `orders.xhtml`. **Converters** turn strings into types; **validators** (Bean Validation by default) reject bad input. **`f:ajax` re-renders parts of the page without a full reload, and** Facelets templates** share layouts across pages.

## NOTES

- Bean Validation constraints run automatically on bound properties
- `f:ajax execute` = what to submit; `render` = what to redraw
- `@this`, `@form`, `@all` are shortcut ids
- PrimeFaces `p:ajax` extends `f:ajax`

## AJAX, CONVERTER, TEMPLATE

```
<!-- template.xhtml -->
<h:body><header>Shop</header><ui:insert name="content"/></h:body>

<!-- orders.xhtml -->
<ui:composition template="/template.xhtml">
  <ui:define name="content">
    <h:form id="f">
      <h:inputText id="qty" value="#{cart.qty}">
        <f:validateLongRange minimum="1" maximum="99"/>
        <f:ajax event="blur" render="total qtyMsg"/>
      </h:inputText>
      <h:message id="qtyMsg" for="qty"/>
      <h:outputText id="total" value="#{cart.total}">
        <f:convertNumber type="currency" currencySymbol="$"/>
      </h:outputText>
      <h:commandButton value="Buy" action="#{cart.buy}"/>
    </h:form>
  </ui:define>
</ui:composition>
```

## CODE EXPLAINED

- → `ui:insert` / `ui:define` — template slot and its content
- → `f:ajax render` — space-separated ids to redraw
- → `f:convertNumber` — formats on output, parses on input
- → `action="#{cart.buy}"` returning `"done"` → `done.xhtml`

## SPRING EQUIVALENT

- Thymeleaf layouts ( `th:replace` ) ≈ Facelets templates
- AJAX → htmx or a JS front end with REST (Spring 28)

## PREFER / AVOID

- ⇄ ~~`faces-config.xml`~~ navigation rules → implicit outcomes
- ⇄ custom ~~`Validator`~~ per field → Bean Validation annotations
- ⇄ full reloads for small updates → `f:ajax`

## NAVIGATION

| OUTCOME | RESULT |
|---|---|
| `"orders"` | forward to `orders.xhtml` |
| `"orders?faces-redirect=true"` | 302 + GET |
| `null / void` | stay, re-render |
| `h:link outcome="x"` | GET link to `x.xhtml` |

## CONVERTERS & VALIDATORS

| TAG | DOES |
|---|---|
| `f:convertDateTime` | `LocalDate`, patterns |
| `f:convertNumber` | currency, percent |
| `f:validateBean` | groups for Bean Validation |
| `f:validateRegex` | pattern match |
| `@FacesConverter` | custom type |

## ⚠ COMMON MISTAKES

- `render` id without the form prefix from outside the form — use `:f:total` or `@form`
- Forward after POST — refresh resubmits the form; add `faces-redirect=true`
- Converter on an input but not the output (or vice versa) — values round-trip inconsistently

## ✓ TIPS & BEST PRACTICES

- Use `@FacesConverter(forClass = Sku.class, managed = true)` to inject into converters
- OmniFaces `o:validateAll` for cross-field checks
- Keep one template per layout; pages only define content

## QUICK SUMMARY

- `outcome` — view id to show
- `faces-redirect` — PRG pattern
- `f:ajax` — partial update
- `ui:composition` — templating

# WebSocket



## DESCRIPTION

**Jakarta WebSocket** gives a full-duplex connection between browser and server over one TCP socket. Annotate a class with **`@ServerEndpoint`** and handle lifecycle callbacks. Each connection gets a `Session` you use to send messages; broadcasting means looping over open sessions.

## NOTES

- A new endpoint instance per connection by default
- Path params: `@ServerEndpoint("/chat/{room}")`
- Encoders/decoders map messages ↔ Java objects
- `getAsyncRemote()` doesn't block the sender

## CHAT ROOM ENDPOINT

```
@ServerEndpoint(value = "/chat/{room}",
                encoders = ChatJson.class, decoders = ChatJson.class)
public class ChatEndpoint {
    private static final Set<Session> sessions =
        ConcurrentHashMap.newKeySet();

    @OnOpen
    public void open(Session s, @PathParam("room") String room) {
        s.getUserProperties().put("room", room);
        sessions.add(s);
    }
    @OnMessage
    public void onMessage(ChatMsg msg, Session from) {
        sessions.stream()
            .filter(s -> s.isOpen() && room(s).equals(room(from)))
            .forEach(s -> s.getAsyncRemote().sendObject(msg));
    }
    @OnClose public void close(Session s) { sessions.remove(s); }
    @OnError public void error(Session s, Throwable t) { log(t); }
}
```

## CODE EXPLAINED

- → `static` set — shared across per-connection instances
- → `getUserProperties()` — per-session data map
- → `sendObject` — runs the encoder; `sendText` for raw strings
- → Browser: `new WebSocket("wss://host/shop/chat/java")`

## SPRING EQUIVALENT

- `@ServerEndpoint` works in Boot with `ServerEndpointExporter`
- Spring's own: `WebSocketHandler` or STOMP `@MessageMapping`

## PREFER / AVOID

- ⇄ ~~`getBasicRemote()`~~ in broadcasts → `getAsyncRemote()`
- ⇄ plain ~~`HashSet`~~ of sessions → `ConcurrentHashMap.newKeySet()`
- ⇄ polling every second → WebSocket or SSE

## CALLBACKS

| ANNOTATION | PARAMS |
|---|---|
| `@OnOpen` | `Session`, `EndpointConfig`, `@PathParam` |
| `@OnMessage` | `String` / object / `ByteBuffer`, `Session` |
| `@OnClose` | `Session`, `CloseReason` |
| `@OnError` | `Session`, `Throwable` |

## SESSION API

| METHOD | DOES |
|---|---|
| `getBasicRemote().sendText()` | blocking send |
| `getAsyncRemote().sendObject()` | non-blocking send |
| `close(CloseReason)` | close with code |
| `setMaxIdleTimeout(ms)` | idle limit |

## ⚠ COMMON MISTAKES

- Non-thread-safe session collection — concurrent opens/closes throw `ConcurrentModificationException`
- Blocking sends in a loop — one slow client stalls everyone
- Forgetting `wss://` behind HTTPS — browsers block mixed content

## ✓ TIPS & BEST PRACTICES

- Send heartbeats ( `sendPing` ) so proxies don't drop idle sockets
- Use a `ServerEndpointConfig.Configurator` to read the HTTP session or auth
- For one-way server pushes, Server-Sent Events via Jakarta REST are simpler

## QUICK SUMMARY

- `@ServerEndpoint` — endpoint path
- `@OnMessage` — handle frames
- `Session` — send / close
- `Encoder/Decoder` — object mapping

# REST Resources



## DESCRIPTION

**Jakarta REST** (formerly JAX-RS) maps HTTP requests to Java methods. An `Application` subclass with **`@ApplicationPath` turns it on; each** resource class** has a `@Path` and methods annotated with an HTTP verb. Return a Java object and a provider (JSON-B) writes the body based on **`@Produces`**.

## NOTES

- REST 4.0 (EE 11) removed the old `ManagedBean` and JAXB coupling
- Resources are CDI beans; inject freely

## UNDER THE CONTAINER

- Resource classes are **request-scoped** by default — a new instance per request
- The runtime picks a `MessageBodyWriter` by return type + `@Produces` + `Accept`
- No match on `@Consumes` → 415; no match on `@Produces` → 406

## CRUD RESOURCE

```
@ApplicationPath("/api")
public class RestApp extends Application { }

@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ProductResource {
    @Inject ProductRepository repo;

    @GET public List<Product> all() { return repo.findAll(); }
    @GET @Path("{id}")
    public Product one(@PathParam("id") long id) {
        return repo.findById(id).orElseThrow(NotFoundException::new);
    }
    @POST public Response create(Product p, @Context UriInfo uri) {
        repo.insert(p);
        URI loc = uri.getAbsolutePathBuilder()
            .path(String.valueOf(p.id())).build();
        return Response.created(loc).build();                  // 201
    }
    @DELETE @Path("{id}") public void delete(@PathParam("id") long id) {
        repo.deleteById(id);                                   // 204
    }
}
```

## CODE EXPLAINED

- → Class-level `@Produces` / `@Consumes` apply to every method
- → `NotFoundException` → 404 automatically
- → `Response.created(uri)` → 201 + `Location` header
- → `void` return → 204 No Content

## SPRING EQUIVALENT

- `@Path` + `@GET` → `@RestController` + `@GetMapping`
- `@PathParam` → `@PathVariable`
- Spring also runs Jersey via `spring-boot-starter-jersey` · See Spring 20

## PREFER / AVOID

- ⇄ returning `Response` everywhere → return the entity; use `Response` for 201/headers
- ⇄ string-built JSON → return records/DTOs

## ANNOTATIONS

| ANNOTATION | PURPOSE |
|---|---|
| `@Path` | URI template, `{id: \\d+}` regex |
| `@GET @POST @PUT…` | HTTP method |
| `@Produces` | response media types |
| `@Consumes` | request media types |

## SUB-RESOURCES

| PATTERN | EXAMPLE |
|---|---|
| `sub-resource method` | `@GET @Path("{id}/reviews")` |
| `sub-resource locator` | `@Path("{id}/reviews") ReviewResource r()` |

## ⚠ COMMON MISTAKES

- Missing `@ApplicationPath` (and no `web.xml` mapping) — every endpoint returns 404
- Two methods with the same path + verb + media type — ambiguous, request fails

## ✓ TIPS & BEST PRACTICES

- Constrain ids with regex: `@Path("{id: \\d+}")` to avoid route clashes
- Use `@Context UriInfo` to build links instead of hard-coding hosts
- See the HTTP & REST APIs set for status codes and design

## QUICK SUMMARY

- `@ApplicationPath` — enables REST
- `@Path` — URI template
- `@GET/@POST…` — verb
- `@Produces` — response type

# Parameters, Responses & Errors



## DESCRIPTION

Jakarta REST injects request data into method parameters by annotation and converts strings to Java types. **`Response` builds any status, header or entity. Exceptions become responses: throw a `WebApplicationException` subclass for HTTP errors, or register an `ExceptionMapper`** to translate your own exceptions.

## NOTES

- `@DefaultValue` applies when the param is missing
- Types convert via `valueOf(String)`, `fromString` or a String ctor
- `List<String>` collects repeated query params

## PARAMS, RESPONSE, MAPPER

```
@GET
public Response search(@QueryParam("q") String q,
                       @QueryParam("page") @DefaultValue("0") int page,
                       @HeaderParam("Accept-Language") String lang,
                       @BeanParam Paging paging) {
    var hits = service.search(q, page);
    return Response.ok(hits)
        .header("X-Total-Count", hits.total())
        .build();                                          // 200
}

public class Paging {                              // @BeanParam holder
    @QueryParam("size") @DefaultValue("20") public int size;
    @QueryParam("sort") public String sort;
}

@Provider
public class MissingMapper
        implements ExceptionMapper<ProductMissing> {
    public Response toResponse(ProductMissing e) {
        return Response.status(404)
            .entity(Map.of("error", e.getMessage())).build();
    }
}
```

## CODE EXPLAINED

- → `@BeanParam` — groups related params into one class
- → `Response.ok(entity)` — 200 + body; chain headers
- → `@Provider` — registers the mapper automatically
- → Mapper picks the closest exception type in the hierarchy

## SPRING EQUIVALENT

- `@QueryParam` → `@RequestParam`; `@HeaderParam` → `@RequestHeader`
- `ExceptionMapper` → `@RestControllerAdvice` + `ProblemDetail` · See Spring 24

## PREFER / AVOID

- ⇄ ~~`try/catch`~~ in every method → one `ExceptionMapper`
- ⇄ ~~`Response.status(404)`~~ inline → `throw new NotFoundException()`
- ⇄ ad-hoc error JSON → RFC 9457 problem details shape

## PARAMETER SOURCES

| ANNOTATION | FROM |
|---|---|
| `@PathParam` | URI template {x} |
| `@QueryParam` | ?x=1 |
| `@HeaderParam` | request header |
| `@CookieParam` | cookie |
| `@FormParam` | form body |

## BUILT-IN EXCEPTIONS

| EXCEPTION | STATUS |
|---|---|
| `BadRequestException` | 400 |
| `NotAuthorizedException` | 401 |
| `ForbiddenException` | 403 |
| `NotFoundException` | 404 |
| `InternalServerErrorException` | 500 |

## ⚠ COMMON MISTAKES

- `int` query param with no default — missing param becomes 0, hiding bugs; use `Integer` or `@DefaultValue`
- Bad conversion on `@PathParam` returns 404, on `@QueryParam` 404 too — not 400 as many expect
- Mapper for `Exception` catches `WebApplicationException` too — keep their status

## ✓ TIPS & BEST PRACTICES

- Return `Optional`-free domain objects; map absence to `NotFoundException`
- Validate params with `@Valid` /constraints (sheet 31)
- Use `UriInfo.getQueryParameters()` for dynamic filters

## QUICK SUMMARY

- `@QueryParam` — query string
- `@BeanParam` — param holder
- `Response` — full control
- `ExceptionMapper` — exception → HTTP

# JSON Processing & Binding



## DESCRIPTION

**JSON-B** binds Java objects to JSON and back, the default provider behind Jakarta REST. **JSON-P** is the lower-level API: build and read JSON trees or stream tokens, plus `JsonPointer` and `JsonPatch`. Use JSON-B for DTOs; reach for JSON-P for dynamic shapes and partial updates.

## NOTES

- Records serialize by component (JSON-B 3)
- Default naming is Java property names; change with a strategy
- Implementations: Yasson (JSON-B), Parsson (JSON-P)
- Jackson is common too, but isn't a Jakarta spec

## JSON-B BINDING

```
public record ProductDto(
    long id,
    @JsonbProperty("display_name") String name,
    @JsonbNumberFormat("#0.00") BigDecimal price,
    @JsonbDateFormat("yyyy-MM-dd") LocalDate added,
    @JsonbTransient String internalNote) { }

Jsonb jsonb = JsonbBuilder.create(new JsonbConfig()
    .withPropertyNamingStrategy(
        PropertyNamingStrategy.LOWER_CASE_WITH_UNDERSCORES)
    .withNullValues(false));
String json = jsonb.toJson(dto);
    // {"id":1,"display_name":"Mug","price":"9.50",...}
ProductDto back = jsonb.fromJson(json, ProductDto.class);
```

## CODE EXPLAINED

- → `@JsonbProperty` — rename one field
- → `@JsonbTransient` — never serialized
- → `withNullValues(false)` — omit nulls
- → `JsonPatch` — RFC 6902 ops: add, remove, replace, move

## SPRING EQUIVALENT

- Spring uses Jackson: `@JsonProperty`, `@JsonIgnore`, `@JsonFormat`
- `ObjectMapper` / Jackson 3 `JsonMapper` ≈ `Jsonb` · See Spring 25

## JSON-P TREE & PATCH

```
JsonObject o = Json.createObjectBuilder()
    .add("sku", "MUG-1").add("qty", 2).build();  // {"sku":"MUG-1","qty":2}
JsonPatch patch = Json.createPatchBuilder()
    .replace("/qty", 5).build();
JsonObject updated = patch.apply(o);          // {"sku":"MUG-1","qty":5}
```

## PREFER / AVOID

- ⇄ serializing entities → record DTOs
- ⇄ ~~`new Jsonb`~~ per call → one shared `Jsonb` (thread-safe)
- ⇄ ~~`Map<String,Object>`~~ for known shapes → typed records

## JSON-B ANNOTATIONS

| ANNOTATION | DOES |
|---|---|
| `@JsonbProperty` | JSON name |
| `@JsonbTransient` | skip |
| `@JsonbDateFormat` | date pattern |
| `@JsonbNillable` | write nulls |
| `@JsonbTypeAdapter` | custom mapping |
| `@JsonbCreator` | ctor / factory for reading |

## JSON-P CLASSES

| CLASS | USE |
|---|---|
| `JsonObjectBuilder` | build objects |
| `JsonReader` | parse to tree |
| `JsonParser` | streaming tokens |
| `JsonPointer` | `/a/0/b` paths |
| `JsonMergePatch` | RFC 7386 merge |

## ⚠ COMMON MISTAKES

- Returning JPA entities — lazy collections trigger extra queries or fail outside the transaction
- Expecting Jackson annotations to work with JSON-B — they're ignored unless Jackson is the provider
- Creating a `Jsonb` per request — it's expensive; cache it or use a `ContextResolver`

## ✓ TIPS & BEST PRACTICES

- Customize REST's JSON-B with a `ContextResolver<Jsonb>` provider
- Use `JsonMergePatch` for `PATCH` with `application/merge-patch+json`
- Keep DTOs as records; validation annotations work on components

## QUICK SUMMARY

- `JSON-B` — object ↔ JSON
- `@JsonbProperty` — rename
- `JSON-P` — trees & streams
- `JsonPatch` — partial updates

# REST Client, Filters & Providers



## DESCRIPTION

The **REST Client API** calls other HTTP services with a fluent builder. On the server side, **providers** extend the runtime: **filters** see every request/response (auth, CORS, logging), **interceptors** wrap entity reading/writing, and **@NameBinding** limits a filter to annotated resources.

## NOTES

- `Client` is heavy and thread-safe — create once, reuse
- Pre-matching filters run before a resource is chosen
- MicroProfile Rest Client offers a typed interface style

## CLIENT CALL

```
Client client = ClientBuilder.newClient();          // reuse this
Product p = client.target("https://api.example.com")
    .path("products/{id}").resolveTemplate("id", 42)
    .request(MediaType.APPLICATION_JSON)
    .get(Product.class);                            // throws on 4xx/5xx
var later = client.target(url).request()
    .rx().get(Product.class);                       // async
```

## CODE EXPLAINED

- → `resolveTemplate` — fills `{id}` safely (URL-encoded)
- → `get(Product.class)` — reads body; errors throw `WebApplicationException`
- → `rx()` — `CompletionStage` invoker
- → `@Audited` on a method/class → only those get `AuditFilter`

## NAME-BOUND FILTER + CORS

```
@NameBinding @Retention(RUNTIME) public @interface Audited { }

@Provider @Audited
public class AuditFilter implements ContainerRequestFilter {
    public void filter(ContainerRequestContext ctx) {
        log.info(ctx.getMethod() + " " + ctx.getUriInfo().getPath());
    }
}
@Provider
public class Cors implements ContainerResponseFilter {
    public void filter(ContainerRequestContext rq,
                       ContainerResponseContext rs) {
        rs.getHeaders().add("Access-Control-Allow-Origin",
                            "https://shop.app");
    }
}
```

## SPRING EQUIVALENT

- `Client` → `RestClient` / HTTP interface clients (Spring 30–31)
- `ContainerRequestFilter` → `OncePerRequestFilter`; CORS → `@CrossOrigin` (Spring 27)

## PREFER / AVOID

- ⇄ new ~~`Client`~~ per call → one shared client
- ⇄ CORS headers in each resource → one response filter

## PROVIDER TYPES

| INTERFACE | RUNS |
|---|---|
| `ContainerRequestFilter` | before resource method |
| `ContainerResponseFilter` | after resource method |
| `ReaderInterceptor / WriterInterceptor` | around body (de)serialization |
| `ExceptionMapper` | exception → response |

## ORDERING

| TOOL | EFFECT |
|---|---|
| `@Priority(Priorities.AUTHENTICATION)` | 1000 — runs early |
| `@PreMatching` | before routing |
| `abortWith(resp)` | stop and respond |

## ⚠ COMMON MISTAKES

- Creating a `Client` per request — leaks connections and threads
- CORS filter only on responses — preflight `OPTIONS` also needs `Allow-Methods` / `Allow-Headers`
- Reading a `Response` entity twice — the stream is gone; call `bufferEntity()` first

## ✓ TIPS & BEST PRACTICES

- Register client filters with `client.register(...)`
- Set `connectTimeout` and `readTimeout` on the `ClientBuilder`
- Use `@Priority` constants from `Priorities` for predictable order

## QUICK SUMMARY

- `ClientBuilder` — outbound calls
- `ContainerRequestFilter` — pre-processing
- `@NameBinding` — targeted filters
- `@Provider` — auto-register

# Entities & persistence.xml



## DESCRIPTION

**Jakarta Persistence** (JPA) maps Java classes to tables. An **@Entity** needs an `@Id` and a no-arg constructor; fields map to columns by name. A **persistence unit** in `META-INF/persistence.xml` names the datasource and settings. EE 11's Persistence 3.2 adds Java records as embeddables and more Java time types.

## NOTES

- Field access if `@Id` is on a field; property access if on a getter
- Use `jta-data-source` so the server manages connections
- `jakarta.persistence.schema-generation.*` creates tables (dev only)
- See the SQL set for schema design

## ENTITY + EMBEDDABLE

```
@Entity @Table(name = "products",
       uniqueConstraints = @UniqueConstraint(columnNames = "sku"))
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, length = 40) private String sku;
    @Column(precision = 10, scale = 2) private BigDecimal price;
    @Enumerated(EnumType.STRING) private Status status;
    @Embedded private Dimensions size;          // columns in same table
    @Version private long version;              // optimistic locking
    protected Product() { }                     // required by JPA
}
@Embeddable public record Dimensions(int width, int height) { } // 3.2
```

## PERSISTENCE.XML

```
<persistence-unit name="shop" transaction-type="JTA">
  <jta-data-source>java:app/jdbc/shop</jta-data-source>
  <properties>
    <property name="jakarta.persistence.schema-generation.database.action"
              value="create"/>
  </properties>
</persistence-unit>
```

## CODE EXPLAINED

- → `IDENTITY` — DB auto-increment; `SEQUENCE` batches better
- → `EnumType.STRING` — survives enum reordering
- → `@Version` — throws `OptimisticLockException` on stale updates
- → `transaction-type="JTA"` — container transactions

## SPRING EQUIVALENT

- Same `jakarta.persistence` annotations in Spring Data JPA
- `persistence.xml` → `spring.datasource.*` + `spring.jpa.*` · See Spring 36

## PREFER / AVOID

- ~~`EnumType.ORDINAL`~~ → `EnumType.STRING`
- ~~schema generation in prod~~ → Flyway/Liquibase migrations
- ~~`java.util.Date`~~ → `LocalDate` / `Instant`

## ID STRATEGIES

| STRATEGY | NOTES |
|---|---|
| `IDENTITY` | auto-increment; no batch inserts |
| `SEQUENCE` | DB sequence; `allocationSize` batches |
| `UUID` | `@GeneratedValue(strategy = UUID)` (3.1+) |
| `TABLE` | portable, slow |
| `AUTO` | provider decides |

## COLUMN MAPPINGS

| ANNOTATION | USE |
|---|---|
| `@Column` | name, nullable, length, precision |
| `@Lob` | large text/binary |
| `@Transient` | not persisted |
| `@Convert` | custom `AttributeConverter` |
| `@Embedded` | value object columns |

## ⚠ COMMON MISTAKES

- `equals` / `hashCode` on the generated id — new entities (id null) break sets; use a business key
- Public no-arg constructor missing — provider can't instantiate; `protected` is enough
- `schema-generation` `drop-and-create` left on — wipes the database on redeploy

## ✓ TIPS & BEST PRACTICES

- Define the datasource in the server or with `@DataSourceDefinition`
- Use `@Column(nullable=false)` to match DB constraints so errors surface early
- Log SQL with your provider's property ( `hibernate.show_sql` ) during dev

## QUICK SUMMARY

- `@Entity` — mapped class
- `@Id` + `@GeneratedValue` — primary key
- `persistence.xml` — unit + datasource
- `@Version` — optimistic lock

# Relationships & Inheritance



## DESCRIPTION

Relationships map foreign keys to object references. The side with the foreign key is the **owner**; the other side uses **mappedBy **and is ignored for writes.** Cascade **propagates operations to children;** fetch type** decides when related rows load. Inheritance maps a class hierarchy to one or more tables.

## NOTES

- `@ManyToOne` / `@OneToOne` default to EAGER — set LAZY
- `@OneToMany` / `@ManyToMany` default to LAZY
- Keep both sides in sync with helper methods
- `orphanRemoval` deletes children removed from the collection

## ORDER ↔ LINES, MANY-TO-MANY TAGS

```
@Entity public class PurchaseOrder {
    @Id @GeneratedValue Long id;
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<OrderLine> lines = new ArrayList<>();

    public void addLine(OrderLine l) { lines.add(l); l.setOrder(this); }
}

@Entity public class OrderLine {
    @Id @GeneratedValue Long id;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id")              // owner: has the FK
    private PurchaseOrder order;
}

@Entity public class Product {
    @ManyToMany
    @JoinTable(name = "product_tags",
        joinColumns = @JoinColumn(name = "product_id"),
        inverseJoinColumns = @JoinColumn(name = "tag_id"))
    private Set<Tag> tags = new HashSet<>();
}
```

## CODE EXPLAINED

- → `mappedBy = "order"` — field name on the owning side
- → `CascadeType.ALL` — persist/remove lines with the order
- → `addLine` — sets both sides so memory matches the DB
- → `Set` for many-to-many avoids duplicate-row deletes

## SPRING EQUIVALENT

- Identical mappings in Spring Data JPA
- Spring's `@EntityGraph` on repository methods ≈ entity graphs here · See Spring 37

## PREFER / AVOID

- ⇄ EAGER ~~`@ManyToOne`~~ → `fetch = LAZY` + fetch joins
- ⇄ ~~`CascadeType.ALL`~~ on ~~`@ManyToOne`~~ → cascade only parent → child
- ⇄ ~~`List`~~ for ~~`@ManyToMany`~~ → `Set`

## INHERITANCE STRATEGIES

| STRATEGY | TABLES | TRADE-OFF |
|---|---|---|
| `SINGLE_TABLE` | one + discriminator | fast; nullable columns |
| `JOINED` | one per class | normalized; joins |
| `TABLE_PER_CLASS` | one per concrete | unions for polymorphic queries |
| `@MappedSuperclass` | none (fields copied) | no polymorphic queries |

## CASCADE TYPES

| TYPE | PROPAGATES |
|---|---|
| `PERSIST` | `persist()` |
| `MERGE` | `merge()` |
| `REMOVE` | `remove()` |
| `ALL` | everything |

## ⚠ COMMON MISTAKES

- Setting only the inverse side — `mappedBy` side is ignored, so the FK stays null
- `CascadeType.REMOVE` on many-to-many — deleting one product deletes shared tags
- Lazy access after the transaction ends — `LazyInitializationException` (Hibernate)

## ✓ TIPS & BEST PRACTICES

- Default every to-one to LAZY and load what you need per query (sheet 28)
- Use `@MappedSuperclass` for shared audit columns
- Prefer `JOINED` when subclasses add many columns

## QUICK SUMMARY

- `owner` — side with `@JoinColumn`
- `mappedBy` — inverse side
- `cascade` — propagate ops
- `@Inheritance` — hierarchy mapping

# EntityManager & Entity Lifecycle



## DESCRIPTION

The **`EntityManager` is the gateway to the persistence context: the set of managed entities for the current transaction. Changes to managed entities are written automatically at flush (**dirty checking**). Entities move between four states: new, managed, detached and removed.

## NOTES

- Inject with `@PersistenceContext` (or `@Inject` with a producer)
- `find()` returns null; `getReference()` returns a lazy proxy
- `merge()` returns a **new** managed copy

## UNDER THE CONTAINER

- The persistence context is a first-level cache: `find()` twice → one SELECT
- Dirty checking at flush writes changed fields — no `update()` call needed
- Flush happens before queries and at commit; transaction-scoped context ends at commit

## STATE TRANSITIONS

```
@ApplicationScoped @Transactional
public class ProductService {
    @PersistenceContext EntityManager em;

    public Long create(String sku) {
        Product p = new Product(sku);          // NEW
        em.persist(p);                         // → MANAGED
        return p.getId();                      // id after flush/IDENTITY
    }
    public void reprice(Long id, BigDecimal price) {
        Product p = em.find(Product.class, id);   // MANAGED (or null)
        p.setPrice(price);                     // UPDATE at commit
    }
    public Product save(Product detached) {    // from REST, DETACHED
        return em.merge(detached);             // returns managed copy
    }
    public void delete(Long id) {
        em.remove(em.getReference(Product.class, id));  // → REMOVED
    }
}
@Entity @EntityListeners(Audit.class) class Product {
    @PrePersist void onCreate() { createdAt = Instant.now(); }
}
```

## CODE EXPLAINED

- → `persist` — new → managed; INSERT at flush
- → `setPrice` on a managed entity — no save call needed
- → `merge` — copies state onto a managed instance; keep using the return value
- → `@PrePersist` — lifecycle callback before INSERT

## SPRING EQUIVALENT

- `em.persist/merge` → `repository.save()` (Spring 38)
- Dirty checking works the same inside `@Transactional`

## PREFER / AVOID

- ⇄ ~~`em.merge()`~~ on managed entities → just modify them
- ⇄ calling ~~`flush()`~~ everywhere → let commit flush
- ⇄ keeping entities across requests → reload by id

## ENTITY STATES

| STATE | IN CONTEXT? | SAVED? |
|---|---|---|
| new | no | no |
| managed | yes | at flush |
| detached | no | only via `merge` |
| removed | yes, scheduled | DELETE at flush |

## CALLBACKS

| ANNOTATION | WHEN |
|---|---|
| `@PrePersist / @PostPersist` | around INSERT |
| `@PreUpdate / @PostUpdate` | around UPDATE |
| `@PreRemove / @PostRemove` | around DELETE |
| `@PostLoad` | after load / refresh |

## ⚠ COMMON MISTAKES

- Ignoring `merge()`'s return value — further changes go to the detached object and are lost
- Calling `persist()` on a detached entity — `EntityExistsException`
- Using `EntityManager` outside a transaction for writes — `TransactionRequiredException`

## ✓ TIPS & BEST PRACTICES

- Use `getReference()` to set a FK without loading the row
- `em.detach(e)` or `clear()` in batch jobs to keep memory flat
- Batch inserts: flush + clear every N rows

## QUICK SUMMARY

- `persist` — new → managed
- `find` — load by id
- `merge` — reattach copy
- `flush` — write pending changes

# JPQL, Criteria & Fetching



## DESCRIPTION

**JPQL** queries entities and fields, not tables and columns; the provider translates it to SQL. Use **named parameters** for values, **`JOIN FETCH` or** entity graphs **to load relationships in one query, and the** Criteria API** when a query is built dynamically. Persistence 3.2 adds `union`, `||`, `cast` and `left`/`right` to JPQL.

## NOTES

- `TypedQuery<T>` avoids casts
- `getSingleResult()` throws if 0 or 2+ rows
- Native SQL: `createNativeQuery(sql, Product.class)`

## UNDER THE CONTAINER

- JPQL is translated to SQL for your database's dialect at runtime
- Lazy associations in a loop cause **N+1**: one query, then one per row
- `JOIN FETCH` and entity graphs change the generated SQL to load it all at once

## QUERIES

```
List<Order> recent = em.createQuery(
        "SELECT o FROM PurchaseOrder o JOIN FETCH o.lines " +
        "WHERE o.customer.email = :email ORDER BY o.placed DESC",
        PurchaseOrder.class)
    .setParameter("email", email)
    .setFirstResult(0).setMaxResults(20)        // page 1, size 20
    .getResultList();

@NamedQuery(name = "Product.bySku",
            query = "SELECT p FROM Product p WHERE p.sku = :sku")

EntityGraph<?> g = em.getEntityGraph("Order.withLines");
em.find(PurchaseOrder.class, id,
        Map.of("jakarta.persistence.fetchgraph", g));

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Product> q = cb.createQuery(Product.class);
Root<Product> p = q.from(Product.class);
q.where(cb.lt(p.get("price"), max), cb.like(p.get("sku"), "MUG%"));
List<Product> cheap = em.createQuery(q).getResultList();
```

## CODE EXPLAINED

- → `:email` — named parameter, never string-concatenate
- → `setFirstResult/setMaxResults` — offset pagination
- → `fetchgraph` — load exactly the graph's attributes
- → Criteria — type-safe with the generated metamodel (`Product_.price`)

## SPRING EQUIVALENT

- JPQL in `@Query` on Spring Data repositories (Spring 40)
- `@EntityGraph` on repository methods; `Specification` ≈ Criteria (Spring 43–44)

## PREFER / AVOID

- ⇄ string concatenation into JPQL → named parameters
- ⇄ ~~`JOIN FETCH`~~ + pagination on collections → two queries or entity graph
- ⇄ ~~`p.get("price")`~~ strings → metamodel `Product_.price`

## JPQL CHEAT LINES

| JPQL | MEANING |
|---|---|
| `SELECT NEW com.x.Dto(p.id, p.sku)` | constructor projection |
| `LEFT JOIN o.lines l` | outer join |
| `WHERE p.tags IS EMPTY` | empty collection |
| `GROUP BY … HAVING COUNT(l) > 2` | aggregates |
| `UPDATE Product p SET …` | bulk update (skips context) |

## RESULT METHODS

| METHOD | RETURNS |
|---|---|
| `getResultList()` | `List<T>` |
| `getSingleResult()` | one or exception |
| `getResultStream()` | `Stream<T>` |
| `executeUpdate()` | rows changed |

## ⚠ COMMON MISTAKES

- `JOIN FETCH` a collection with `setMaxResults` — the provider paginates in memory (and warns)
- Using table/column names in JPQL — it uses entity and field names
- Bulk `UPDATE` / `DELETE` then reading managed entities — the context still holds old values; `clear()`

## ✓ TIPS & BEST PRACTICES

- Enable SQL logging and count queries per request to catch N+1
- Use DTO projections for read-only lists
- Validate named queries at deploy: errors surface early

## QUICK SUMMARY

- `JPQL` — entity-level queries
- `JOIN FETCH` — eager load per query
- `EntityGraph` — reusable fetch plan
- `Criteria` — dynamic queries

# Jakarta Data Repositories



## DESCRIPTION

**Jakarta Data** (new in EE 11) generates data access from an annotated interface, much like Spring Data. Extend `BasicRepository` or `CrudRepository`, or write **annotated methods** (`@Find`, `@Insert`, `@Delete`) and **`@Query`** with JDQL. The runtime implements the interface; inject it with `@Inject`.

## NOTES

- Works over Jakarta Persistence or NoSQL providers
- Method-name queries ( `findByName` ) exist but annotations are preferred
- `@By` matches a parameter to an attribute
- Hibernate generates repositories at compile time

## REPOSITORY INTERFACE

```
@Repository
public interface Products extends CrudRepository<Product, Long> {

    @Find Optional<Product> bySku(@By("sku") String sku);

    @Find @OrderBy("price")
    List<Product> inStatus(Status status);           // by param name

    @Query("WHERE price < :max AND name LIKE :pattern")
    Page<Product> cheap(BigDecimal max, String pattern, PageRequest page);

    @Delete void discontinued(Status status);        // bulk delete

    List<Product> findByNameContainsIgnoreCase(String part);  // name-based
}

@Inject Products products;
products.insert(new Product("MUG-1"));
Page<Product> first = products.cheap(TEN, "%mug%",
        PageRequest.ofSize(20));                     // page 1
first.content(); first.totalElements(); first.hasNext();
```

## CODE EXPLAINED

- → `CrudRepository` — `insert`, `update`, `save`, `findById`, `delete` …
- → `@Find` — parameters become `WHERE` conditions
- → `@Query` — JDQL, a subset of JPQL; `FROM` optional
- → `PageRequest` — 1-based pages, offset or cursor

## SPRING EQUIVALENT

- `CrudRepository` → Spring Data `CrudRepository` (Spring 38)
- `@Query` / derived names → same idea (Spring 39–40)
- `PageRequest.ofSize` → `PageRequest.of(page, size)`

## PREFER / AVOID

- ⇄ hand-written DAO classes → `@Repository` interface
- ⇄ long method names → `@Find` + `@By`
- ⇄ ~~`save()`~~ everywhere → explicit `insert` / `update`

## LIFECYCLE ANNOTATIONS

| ANNOTATION | DOES |
|---|---|
| `@Insert` | persist new entity |
| `@Update` | update existing |
| `@Save` | upsert |
| `@Delete` | delete by entity or params |
| `@Find` | query by params |

## PAGING & SORTING

| TYPE | USE |
|---|---|
| `PageRequest.ofPage(2)` | offset page |
| `PageRequest.afterCursor(…)` | keyset paging |
| `Order.by(Sort.asc("name"))` | dynamic sort |
| `@OrderBy("price")` | static sort |
| `Limit.of(10)` | first N |

## ⚠ COMMON MISTAKES

- Mixing `insert` with an existing id — throws `EntityExistsException`; use `update` or `save`
- Parameter names without `-parameters` compiler flag — `@Find` can't match; add `@By` or the flag
- Expecting EE 10 servers to support it — Jakarta Data needs an EE 11 runtime

## ✓ TIPS & BEST PRACTICES

- Prefer cursor paging for large or changing tables
- Return `Stream<T>` for big reads and close it
- Keep complex queries in `@Query`; method names don't scale

## QUICK SUMMARY

- `@Repository` — generated interface
- `@Find` / `@Query` — reads
- `@Insert/@Update/@Delete` — writes
- `PageRequest` — paging

# Transactions



## DESCRIPTION

Jakarta EE uses **JTA** transactions managed by the container. Put **`@Transactional` on a CDI bean class or method and the interceptor starts, joins or suspends a transaction based on** `TxType`**. EJBs are transactional by default (`@TransactionAttribute`). Use **`UserTransaction`** only for manual control.

## NOTES

- `@Transactional` from `jakarta.transaction`, not Spring's
- Default type is `REQUIRED`
- Persistence `EntityManager` joins the JTA transaction

## UNDER THE CONTAINER

- JTA binds the transaction to the current thread
- `REQUIRED` joins an existing transaction; `REQUIRES_NEW` suspends it and starts another
- Unchecked exceptions roll back; checked exceptions commit unless listed in `rollbackOn`

## DECLARATIVE AND MANUAL

```
@ApplicationScoped
public class PaymentService {
    @Inject AuditLog audit;
    @PersistenceContext EntityManager em;

    @Transactional(rollbackOn = PaymentDeclined.class)
    public void charge(Order o) throws PaymentDeclined {
        em.persist(new Payment(o));
        audit.record("charge " + o.id());   // runs in its own tx
        gateway.charge(o);                  // may throw
    }
}
@ApplicationScoped
public class AuditLog {
    @Transactional(Transactional.TxType.REQUIRES_NEW)
    public void record(String msg) { em.persist(new AuditEntry(msg)); }
}

@Resource UserTransaction utx;              // manual (rare)
utx.begin();
try { work(); utx.commit(); }
catch (Exception e) { utx.rollback(); throw e; }
```

## CODE EXPLAINED

- → `rollbackOn` — make a checked exception roll back
- → `REQUIRES_NEW` — audit entry commits even if `charge` rolls back
- → Must call `record` through the injected bean, not `this`
- → `UserTransaction` — bean-managed; don't mix with `@Transactional`

## SPRING EQUIVALENT

- `@Transactional` → Spring `@Transactional` ( `rollbackFor`, `propagation` )
- `TxType` names match Spring `Propagation` · See Spring 42

## PREFER / AVOID

- ⇄ ~~`@Stateless`~~ just for transactions → CDI + `@Transactional`
- ⇄ ~~`UserTransaction`~~ boilerplate → `@Transactional`
- ⇄ transactions on REST resources → on the service layer

## TXTYPE

| TYPE | NO TX EXISTS | TX EXISTS |
|---|---|---|
| `REQUIRED` | start new | join |
| `REQUIRES_NEW` | start new | suspend, start new |
| `MANDATORY` | throw | join |
| `SUPPORTS` | run without | join |
| `NOT_SUPPORTED` | run without | suspend |

## EJB EQUIVALENT

| EJB | CDI |
|---|---|
| `@TransactionAttribute` | `@Transactional` |
| `setRollbackOnly()` | same, on `TransactionManager` |
| `@ApplicationException` | `rollbackOn` |

## ⚠ COMMON MISTAKES

- Importing Spring's `@Transactional` in a Jakarta app — nothing intercepts it
- Self-invocation of a `REQUIRES_NEW` method — runs in the caller's transaction
- Catching the exception inside the method — the interceptor never sees it, so it commits

## ✓ TIPS & BEST PRACTICES

- Keep transactions short; no remote calls inside if you can help it
- Use `@Transactional(dontRollbackOn = …)` for expected business exceptions
- Observe `AFTER_SUCCESS` events for work that must follow a commit

## QUICK SUMMARY

- `@Transactional` — CDI interceptor
- `REQUIRED` — join or start
- `REQUIRES_NEW` — independent tx
- `rollbackOn` — checked rollback

# Bean Validation



## DESCRIPTION

**Jakarta Validation** declares rules as annotations on fields, getters, parameters, return values and record components. REST, Faces, Persistence and CDI run validation automatically; elsewhere call a `Validator`. **`@Valid` cascades into nested objects;** groups** select which constraints apply.

## NOTES

- REST returns 400 on violations of `@Valid` parameters
- JPA validates on persist/update by default
- Validation 3.1 (EE 11) supports records

## CONSTRAINTS, CASCADE, CUSTOM

```
public record SignupForm(
    @NotBlank @Size(max = 50) String name,
    @NotNull @Email String email,
    @Min(18) int age,
    @Valid @NotNull Address address,              // validate nested
    @Sku String favouriteSku) { }

@POST public Response signup(@Valid SignupForm form) { ... }  // 400 if bad

@Constraint(validatedBy = SkuValidator.class)
@Target({FIELD, PARAMETER, RECORD_COMPONENT}) @Retention(RUNTIME)
public @interface Sku {
    String message() default "invalid SKU";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
public class SkuValidator implements ConstraintValidator<Sku, String> {
    public boolean isValid(String v, ConstraintValidatorContext c) {
        return v == null || v.matches("[A-Z]{3}-\\d+");   // null = valid
    }
}

Set<ConstraintViolation<SignupForm>> errs = validator.validate(form);
```

## CODE EXPLAINED

- → `@NotBlank` — not null and not whitespace (strings only)
- → `@Valid` on a param — REST/CDI validate before the method runs
- → Custom constraint = annotation + `ConstraintValidator`
- → Return `true` for null and pair with `@NotNull` if required

## SPRING EQUIVALENT

- Spring uses the same `jakarta.validation` annotations
- `@Valid` on `@RequestBody`; `@Validated` for method validation · See Spring 23

## PREFER / AVOID

- ⇄ ~~`if (x == null) throw`~~ checks → constraint annotations
- ⇄ validation logic in controllers → on the DTO
- ⇄ one-off ~~`@Pattern`~~ repeated → custom constraint

## BUILT-IN CONSTRAINTS

| ANNOTATION | CHECKS |
|---|---|
| `@NotNull / @NotEmpty / @NotBlank` | null / empty / blank |
| `@Size(min, max)` | length / count |
| `@Min @Max @Positive` | numbers |
| `@Past @Future` | dates |
| `@Pattern(regexp)` | regex |
| `@Email` | email format |

## GROUPS & MESSAGES

| FEATURE | EXAMPLE |
|---|---|
| `groups` | `groups = OnUpdate.class` |
| `validate group` | `validate(o, OnUpdate.class)` |
| `message` | `message = "{sku.invalid}"` |
| `bundle` | `ValidationMessages.properties` |

## ⚠ COMMON MISTAKES

- `@NotNull` on a primitive `int` — never null, so it never fails; use `@Min` or `Integer`
- Forgetting `@Valid` on a nested object — its constraints are skipped
- Throwing from `isValid` — return `false` and let the framework build the violation

## ✓ TIPS & BEST PRACTICES

- Map `ConstraintViolationException` to a clean 400 body with an `ExceptionMapper`
- Use `@AssertTrue` methods for cross-field rules
- Put messages in `ValidationMessages.properties` for i18n

## QUICK SUMMARY

- `@NotNull` etc. — declarative rules
- `@Valid` — cascade / trigger
- `groups` — conditional rules
- `ConstraintValidator` — custom logic

# Security



## DESCRIPTION

**Jakarta Security** lets an app define **authentication mechanisms** (Basic, Form, OpenID Connect) and **identity stores** (database, LDAP, custom) in code, without server-specific config. **`@RolesAllowed` protects CDI beans, EJBs and REST resources; `SecurityContext`** answers who the caller is.

## NOTES

- Security 4.0 (EE 11) supports multiple mechanisms with qualifiers
- Roles must be declared ( `@DeclareRoles` ) or mapped
- `@PermitAll` / `@DenyAll` override class-level rules

## BASIC AUTH + DB IDENTITY STORE

```
@BasicAuthenticationMechanismDefinition(realmName = "shop")
@DatabaseIdentityStoreDefinition(
    dataSourceLookup = "java:app/jdbc/shop",
    callerQuery = "SELECT password FROM users WHERE email = ?",
    groupsQuery = "SELECT role FROM user_roles WHERE email = ?",
    hashAlgorithm = Pbkdf2PasswordHash.class)
@DeclareRoles({"admin", "customer"})
@ApplicationScoped public class SecurityConfig { }

@Path("/admin") @RolesAllowed("admin")
public class AdminResource {
    @Inject SecurityContext security;

    @GET public String me() {
        return security.getCallerPrincipal().getName();   // bob@shop.app
    }
    @GET @Path("public") @PermitAll public String ping() { return "ok"; }
}

@OpenIdAuthenticationMechanismDefinition(
    providerURI = "https://login.example.com",
    clientId = "shop", clientSecret = "${oidc.secret}",
    redirectURI = "${baseURL}/callback")
```

## CODE EXPLAINED

- → Mechanism + identity store = complete login with no `web.xml`
- → `Pbkdf2PasswordHash` — salted hashing for stored passwords
- → `isCallerInRole("admin")` — programmatic check
- → OIDC mechanism handles the redirect/callback flow

## SPRING EQUIVALENT

- Spring Security `SecurityFilterChain` replaces mechanisms/stores (Spring 45–46)
- `@RolesAllowed` works with `@EnableMethodSecurity(jsr250Enabled = true)`

## PREFER / AVOID

- ⇄ ~~`<security-constraint>`~~ in ~~`web.xml`~~ → `@RolesAllowed`
- ⇄ plain-text passwords → `Pbkdf2PasswordHash`
- ⇄ server-specific realms → `@…IdentityStoreDefinition`

## MECHANISMS

| ANNOTATION | FLOW |
|---|---|
| `@BasicAuthenticationMechanismDefinition` | HTTP Basic |
| `@FormAuthenticationMechanismDefinition` | login page |
| `@CustomFormAuthenticationMechanismDefinition` | your Faces/JSP form |
| `@OpenIdAuthenticationMechanismDefinition` | OIDC login |

## CHECKS

| API | USE |
|---|---|
| `@RolesAllowed("x")` | declarative |
| `SecurityContext.isCallerInRole` | programmatic |
| `getCallerPrincipal()` | current user |
| `authenticate(req, res, params)` | trigger login |

## ⚠ COMMON MISTAKES

- `@RolesAllowed` on a class nobody intercepts (plain `new` object) — no check happens
- Basic auth over HTTP — credentials are only Base64; always TLS
- Mixing REST's `jakarta.ws.rs.core.SecurityContext` and Jakarta Security's — different APIs

## ✓ TIPS & BEST PRACTICES

- Implement `IdentityStore` for custom user sources
- Use `HttpAuthenticationMechanism` for token/JWT auth (or MicroProfile JWT)
- Test with `@RunAs` or runtime test utilities

## QUICK SUMMARY

- `mechanism` — how credentials arrive
- `IdentityStore` — validates them
- `@RolesAllowed` — authorization
- `SecurityContext` — caller info

# Concurrency



## DESCRIPTION

**Jakarta Concurrency** gives you executors that the server manages and that carry the **container context** (security, naming, CDI) into the task. EE 11's 3.1 adds **virtual threads** (`virtual = true`), CDI `@Asynchronous` with scheduling, and injectable executors via qualifiers.

## NOTES

- Never create raw threads in a server app
- Every server has a default `ManagedExecutorService`
- `@Asynchronous` methods return `CompletableFuture`

## UNDER THE CONTAINER

- Managed executors propagate security, naming and CDI context that a raw `ExecutorService` drops
- The server tracks and stops managed tasks at undeploy
- `virtual = true` runs tasks on virtual threads (Java 21+)

## EXECUTORS AND @ASYNCHRONOUS

```
@ManagedExecutorDefinition(name = "java:app/concurrent/io",
                           maxAsync = 50, virtual = true)
@ApplicationScoped
public class ReportService {
    @Resource(lookup = "java:app/concurrent/io")
    ManagedExecutorService io;

    public CompletableFuture<Report> build(long id) {
        return io.supplyAsync(() -> render(id));     // context propagated
    }

    @Asynchronous(executor = "java:app/concurrent/io")
    public CompletableFuture<Void> email(long id) {
        mailer.send(id);
        return Asynchronous.Result.complete(null);
    }

    @Asynchronous(runAt = @Schedule(cron = "0 2 * * *"))   // 3.1
    public void nightly() { purgeOld(); }
}
```

## CODE EXPLAINED

- → `@ManagedExecutorDefinition` — declare an executor in code
- → `supplyAsync` — managed `CompletableFuture`
- → `runAt = @Schedule` — cron scheduling without EJB

## SPRING EQUIVALENT

- `@Asynchronous` → `@Async` (Spring 62)
- `virtual = true` → `spring.threads.virtual.enabled=true`
- `runAt` → `@Scheduled(cron = …)` (Spring 61)

## PREFER / AVOID

- ⇄ ~~`new Thread()`~~ / ~~`Executors.newFixedThreadPool`~~ → managed executor
- ⇄ EJB ~~`@Asynchronous`~~ → Concurrency `@Asynchronous`
- ⇄ EJB timers without persistence → `@Asynchronous(runAt)`

## RESOURCE TYPES

| TYPE | USE |
|---|---|
| `ManagedExecutorService` | async tasks |
| `ManagedScheduledExecutorService` | delayed / periodic |
| `ManagedThreadFactory` | threads for libraries |
| `ContextService` | wrap tasks with context |

## DEFINITION ATTRIBUTES

| ATTRIBUTE | MEANING |
|---|---|
| `maxAsync` | concurrency limit |
| `virtual` | virtual threads |
| `context` | which contexts propagate |
| `qualifiers` | inject with `@Inject @MyQualifier` |

## ⚠ COMMON MISTAKES

- Using `ForkJoinPool.commonPool()` (`CompletableFuture.supplyAsync` without executor) — tasks lose security and CDI context
- Using `@RequestScoped` beans in async tasks — the request is over; pass data in
- Blocking on `.get()` right after submitting — no concurrency gained

## ✓ TIPS & BEST PRACTICES

- Use qualifiers (3.1) to inject executors without JNDI strings
- Cap `maxAsync` to protect downstream systems
- Prefer virtual threads for blocking I/O work

## QUICK SUMMARY

- `ManagedExecutorService` — server-managed pool
- `@Asynchronous` — async CDI method
- `virtual = true` — virtual threads
- `runAt` — scheduled method

# Messaging



## DESCRIPTION

**Jakarta Messaging** (JMS) sends messages through a broker. A **queue** delivers each message to one consumer; a **topic** delivers to every subscriber. Send with the simplified **`JMSContext` API; consume with a** message-driven bean**, which the container pools and wraps in a transaction.

## NOTES

- Platform profile only
- Brokers: ActiveMQ Artemis (WildFly, Payara), Liberty built-in
- Messages: `TextMessage`, `ObjectMessage`, `MapMessage` …

## UNDER THE CONTAINER

- Message-driven beans are pooled consumers; many messages process in parallel
- The container acknowledges the message when the MDB's transaction commits
- On rollback the broker redelivers; set a max-redelivery and dead-letter queue

## SEND AND CONSUME

```
@ApplicationScoped
public class OrderPublisher {
    @Inject JMSContext jms;                          // container-managed
    @Resource(lookup = "java:app/jms/orders") Queue orders;

    public void publish(Order o) {
        jms.createProducer()
           .setProperty("priority", o.vip() ? "high" : "normal")
           .send(orders, jsonb.toJson(o));           // TextMessage
    }
}

@MessageDriven(activationConfig = {
    @ActivationConfigProperty(propertyName = "destinationLookup",
                              propertyValue = "java:app/jms/orders"),
    @ActivationConfigProperty(propertyName = "messageSelector",
                              propertyValue = "priority = 'high'") })
public class VipOrderListener implements MessageListener {
    public void onMessage(Message m) {
        Order o = jsonb.fromJson(((TextMessage) m).getText(), Order.class);
        fulfil(o);                                   // throws → redelivery
    }
}
```

## CODE EXPLAINED

- → `createProducer().send()` — one line to send
- → `messageSelector` — SQL-like filter on properties
- → Exception in `onMessage` → rollback → redelivery

## SPRING EQUIVALENT

- MDB → `@JmsListener` (Spring JMS) or `@KafkaListener`
- `JMSContext` → `JmsTemplate` / `JmsClient`

## PREFER / AVOID

- ⇄ ~~`ObjectMessage` with Java serialization~~ → JSON `TextMessage`
- ⇄ classic ~~`Connection` / `Session`~~ code → `JMSContext`
- ⇄ non-idempotent consumers → handle redelivered duplicates

## QUEUE VS TOPIC

| ASPECT | QUEUE | TOPIC |
|---|---|---|
| `delivery` | one consumer | all subscribers |
| `missed messages` | kept until read | lost unless durable |
| `use for` | work items | notifications |

## ACKNOWLEDGEMENT

| MODE | BEHAVIOR |
|---|---|
| `transacted (MDB default)` | ack on commit |
| `AUTO_ACKNOWLEDGE` | ack after onMessage returns |
| `DUPS_OK_ACKNOWLEDGE` | lazy ack, duplicates possible |
| `CLIENT_ACKNOWLEDGE` | `message.acknowledge()` |

## ⚠ COMMON MISTAKES

- Poison message with no redelivery limit — fails and redelivers forever
- Non-durable topic subscriber — messages sent while it's down are gone

## ✓ TIPS & BEST PRACTICES

- Configure a dead-letter queue and max delivery attempts on the broker
- Make consumers idempotent (dedupe by message id or business key)
- Use `JMSContext.createSharedDurableConsumer` for scaled topic consumers

## QUICK SUMMARY

- `queue` — point-to-point
- `topic` — publish/subscribe
- `JMSContext` — simplified send
- `@MessageDriven` — pooled consumer

# Batch, Mail & Other Specs



## DESCRIPTION

**Jakarta Batch** runs long jobs as **steps**: either **chunk** (read → process → write in transactional chunks, with restart) or **batchlet** (one task). **Jakarta Mail** builds and sends email. Several XML-era specs (XML Binding, XML Web Services) are now optional and ship separately when you need them.

## NOTES

- Batch jobs live in `META-INF/batch-jobs/*.xml`
- Chunk steps checkpoint, so restarts resume
- Mail session is usually a server resource

## BATCH JOB (CHUNK)

```xml
<!-- META-INF/batch-jobs/import-products.xml -->
<job id="import-products" version="2.0">
  <step id="load">
    <chunk item-count="100">
      <reader ref="csvReader"/>
      <processor ref="productProcessor"/>
      <writer ref="productWriter"/>
    </chunk>
  </step>
</job>
```

## CODE EXPLAINED

- → `item-count` — commit every 100 items
- → `ref` — CDI bean names ( `@Named` ) for reader/processor/writer
- → `JobOperator` — start, stop, restart, query executions
- → `Transport.send` — uses the session's SMTP config

## START A JOB · SEND MAIL

```java
long exec = BatchRuntime.getJobOperator()
    .start("import-products", new Properties());

@Resource(lookup = "java:app/mail/shop") Session mail;
MimeMessage m = new MimeMessage(mail);
m.setFrom("shop@example.com");
m.setRecipients(Message.RecipientType.TO, "bob@example.com");
m.setSubject("Order shipped");
m.setText("Your order is on its way.", "UTF-8");
Transport.send(m);
```

## SPRING EQUIVALENT

- Jakarta Batch → Spring Batch ( `Job` , `Step` , `ItemReader` )
- Jakarta Mail → `JavaMailSender` (wraps the same API)

## PREFER / AVOID

- ⇄ hand-rolled import loops → chunk step with restart
- ⇄ SMTP settings in code → server mail resource
- ⇄ new SOAP services → REST + JSON

## BATCH ARTIFACTS

| INTERFACE | ROLE |
|---|---|
| `ItemReader` | `readItem()` → next item or null |
| `ItemProcessor` | transform / filter (null skips) |
| `ItemWriter` | `writeItems(List)` per chunk |
| `Batchlet` | single task `process()` |

## SPEC STATUS (EE 11)

| SPEC | STATUS |
|---|---|
| `XML Binding (JAXB)` | standalone, not in Platform |
| `XML Web Services` | optional / separate |
| `SOAP with Attachments` | standalone |
| `Activation` | standalone |
| `Connectors` | Platform |

## ⚠ COMMON MISTAKES

- Non-restartable reader (no checkpoint) — restart reprocesses from the beginning
- Sending mail inside the order transaction — rollback can't unsend; send after commit
- Assuming JAXB is on the classpath — add `jakarta.xml.bind-api` + an implementation

## ✓ TIPS & BEST PRACTICES

- Return checkpoint info from `checkpointInfo()` for restartable readers
- Observe commit events ( `AFTER_SUCCESS` ) to send mail
- Check `getJobExecution(id).getBatchStatus()` to monitor

## QUICK SUMMARY

- `chunk` — read/process/write in commits
- `batchlet` — single task
- `JobOperator` — control jobs
- `MimeMessage` — email

# Testing Jakarta Apps



## DESCRIPTION

Test Jakarta code in three layers. **Unit tests** construct beans directly with mocks (constructor injection makes this easy). **CDI tests** boot a lightweight container (Weld JUnit) to check wiring. **Integration tests** deploy to a real runtime, via Arquillian or a Testcontainers server image, and hit it over HTTP with REST Assured.

## NOTES

- The JUnit set covers JUnit itself
- Runtimes with dev mode (Liberty, WildFly) also run tests against a live server
- Use Testcontainers for the database even in unit-ish tests

## UNIT, CDI AND INTEGRATION TESTS

```
class PriceServiceTest {                          // plain unit test
    @Test void appliesDiscount() {
        var repo = mock(ProductRepository.class);
        when(repo.price("MUG")).thenReturn(new BigDecimal("10"));
        var svc = new PriceService(repo);          // constructor injection
        assertEquals(new BigDecimal("9.00"), svc.discounted("MUG"));
    }
}

@EnableAutoWeld @AddBeanClasses({PriceService.class, FakeRepo.class})
class PriceWiringTest {                           // real CDI container
    @Inject PriceService svc;
    @Test void wires() { assertNotNull(svc); }
}

@Testcontainers
class ProductApiIT {                              // real server
    @Container static GenericContainer<?> app =
        new GenericContainer<>("shop-wildfly").withExposedPorts(8080);
    @Test void lists() {
        given().port(app.getMappedPort(8080))
            .get("/shop/api/products").then().statusCode(200);
    }
}
```

## CODE EXPLAINED

- → Mock repositories, not the `EntityManager`, where you can
- → `@EnableAutoWeld` — Weld JUnit 5 extension; add beans explicitly
- → `GenericContainer` — your app image with the war deployed
- → REST Assured `given().get().then()` — HTTP assertions

## SPRING EQUIVALENT

- `@SpringBootTest` ≈ integration test; `@WebMvcTest` slices ≈ CDI tests (Spring 52–53)
- Testcontainers works the same in both · See Spring 56

## PREFER / AVOID

- ⇄ Arquillian for every test → unit tests + a few integration tests
- ⇄ mocking ~~`EntityManager`~~ chains → test with a real DB container
- ⇄ field injection in beans under test → constructor injection

## TEST LAYERS

| LAYER | TOOL | SPEED |
|---|---|---|
| unit | JUnit + Mockito | ms |
| CDI wiring | Weld JUnit | ~1 s |
| persistence | Testcontainers DB + JPA | seconds |
| full app | Arquillian / server container | 10s of seconds |

## MAVEN PLUGINS

| PLUGIN | RUNS |
|---|---|
| surefire | *Test in test phase |
| failsafe | *IT in integration-test |
| liberty:dev | tests on Enter, live server |

## ⚠ COMMON MISTAKES

- Mocking `EntityManager` query chains — brittle and proves nothing about the SQL
- Running integration tests with Surefire — they run before the war is built
- Sharing a mutable container between tests without cleanup — order-dependent failures

## ✓ TIPS & BEST PRACTICES

- Make beans constructor-injected so tests need no container
- Reuse containers across a test class with `static`
- Name integration tests `*IT` so Failsafe picks them up

## QUICK SUMMARY

- `unit` — new + mocks
- `Weld JUnit` — CDI wiring
- `Testcontainers` — real DB / server
- `REST Assured` — HTTP checks

# MicroProfile



## DESCRIPTION

**MicroProfile** is an Eclipse spec family built on the Jakarta EE Core Profile for cloud services: config, health checks, fault tolerance, OpenAPI docs, telemetry and a typed REST client. Most Jakarta runtimes (Open Liberty, WildFly, Payara, Helidon, Quarkus) support it alongside EE.

## NOTES

- MicroProfile 7 builds on Jakarta EE 10 Core
- Telemetry (OpenTelemetry) replaced Metrics and OpenTracing
- Config sources: system props > env vars > `microprofile-config.properties`

## CONFIG, HEALTH, FAULT TOLERANCE, REST CLIENT

```
@ApplicationScoped
public class Inventory {
    @Inject @ConfigProperty(name = "inventory.url",
                            defaultValue = "http://localhost:9080")
    String url;

    @Retry(maxRetries = 3, delay = 200)
    @Timeout(2000)
    @CircuitBreaker(requestVolumeThreshold = 10, failureRatio = 0.5)
    @Fallback(fallbackMethod = "cached")
    public int stock(String sku) { return client.stock(sku); }
    int cached(String sku) { return cache.getOrDefault(sku, 0); }
}

@Readiness @ApplicationScoped
public class DbCheck implements HealthCheck {
    public HealthCheckResponse call() {
        return HealthCheckResponse.named("db").status(db.ping()).build();
    }
}

@RegisterRestClient(configKey = "inventory") @Path("/stock")
public interface InventoryClient {
    @GET @Path("{sku}") int stock(@PathParam("sku") String sku);
}
```

## CODE EXPLAINED

- → `@ConfigProperty` — injected, typed config with default
- → `@Retry` + `@Timeout` + `@CircuitBreaker` — stack in that order
- → `@Readiness` — `/health/ready`; `@Liveness` — `/health/live`
- → `@RegisterRestClient` — typed client; inject with `@RestClient`

## SPRING EQUIVALENT

- `@ConfigProperty` → `@Value` / `@ConfigurationProperties` (Spring 14–15)
- Health → Actuator `/actuator/health` (Spring 57)
- Fault Tolerance → `@Retryable`, Resilience4j (Spring 64)

## PREFER / AVOID

- ⇄ config in ~~`web.xml`~~ context params → MicroProfile Config
- ⇄ hand-written retry loops → `@Retry`
- ⇄ manual ~~`Client`~~ code for each API → `@RegisterRestClient`

## SPECS

| SPEC | GIVES YOU |
|---|---|
| Config | `@ConfigProperty`, sources |
| Health | `/health/live`, `/health/ready` |
| Fault Tolerance | retry, timeout, breaker, bulkhead |
| OpenAPI | `/openapi` + `@Operation` |
| Telemetry | OpenTelemetry traces/metrics |
| JWT Auth | `@RolesAllowed` from JWT |

## CONFIG LOOKUP

| SOURCE | ORDINAL |
|---|---|
| System properties | 400 |
| Environment vars | 300 |
| microprofile-config.properties | 100 |
| custom ConfigSource | your choice |

## ⚠ COMMON MISTAKES

- Env var names — `inventory.url` maps to `INVENTORY_URL`, not `inventory_url`
- `@Fallback` method with a different signature — deploy fails
- Using liveness for DB checks — the platform restarts pods during DB outages; use readiness

## ✓ TIPS & BEST PRACTICES

- Expose `/openapi/ui` in dev for a browsable API
- Use profiles: `%dev.inventory.url=…`
- Bulkhead (`@Bulkhead`) caps concurrent calls to slow services

## QUICK SUMMARY

- `Config` — external settings
- `Health` — probes
- `Fault Tolerance` — resilience annotations
- `Rest Client` — typed HTTP client

# Packaging, Containers & What's Next



## DESCRIPTION

Ship a **thin war** onto a runtime image, or build a **bootable jar** that contains a trimmed server. Either way, keep config out of the artifact (env vars, MicroProfile Config) and log to stdout for containers. **Jakarta EE 12** is in milestones now: Core Profile is planned for Dec 2026 and the full Platform for mid-2027.

## NOTES

- Thin war + runtime image layers cache well in Docker
- WildFly Galleon and Liberty features trim the server
- See the Docker and Kubernetes sets for images and probes

## DOCKERFILE — THIN WAR ON OPEN LIBERTY

```
FROM icr.io/appcafe/open-liberty:full-java21-openj9-ubi-minimal
COPY --chown=1001:0 src/main/liberty/config/server.xml /config/
COPY --chown=1001:0 target/shop.war /config/apps/
RUN configure.sh                        # pre-warm, build caches
ENV INVENTORY_URL=http://inventory:9080
EXPOSE 9080
```

## CODE EXPLAINED

- → `configure.sh` — Liberty build step that caches config for fast start
- → Config via env vars → read by MicroProfile Config
- → `layers` — only the subsystems the app needs
- → `java -jar target/shop-bootable.jar` runs it

## BOOTABLE JAR (WILDFLY)

```
<plugin>
  <groupId>org.wildfly.plugins</groupId>
  <artifactId>wildfly-jar-maven-plugin</artifactId>
  <configuration>
    <feature-packs>
      <feature-pack><location>wildfly@maven(...)</location></feature-pack>
    </feature-packs>
    <layers><layer>jaxrs-server</layer></layers>
  </configuration>
</plugin>
```

## SPRING EQUIVALENT

- Bootable jar ≈ Spring Boot executable jar
- `bootBuildImage` ≈ runtime base images (Spring 66)
- GraalVM native: Quarkus/Helidon for Jakarta, Spring native for Boot (Spring 67)

## PREFER / AVOID

- ⇄ config baked into the war → env vars / Config
- ⇄ log files inside containers → stdout + collector
- ⇄ full Platform image for a REST service → trimmed layers/features

## ARTIFACT OPTIONS

| OPTION | PROS | CONS |
|---|---|---|
| `thin war` | small, fast layer rebuilds | needs runtime image |
| `bootable / uber jar` | one file, `java -jar` | bigger artifact |
| `native image` | ms startup, low RAM | build time, reflection config |

## EE 12 (IN PROGRESS)

| AREA | DIRECTION |
|---|---|
| `Core Profile` | planned Dec 2026 |
| `Web Profile` | planned Mar 2027 |
| `Platform` | planned May 2027 |
| `Themes` | consistency, Jakarta Query, more CDI-first APIs |

## ⚠ COMMON MISTAKES

- Rebuilding the whole image for a code change — copy the war last so runtime layers stay cached
- Health checks pointing at `/` — use MicroProfile Health endpoints for probes
- Assuming EE 12 APIs are final — milestone APIs can still change

## ✓ TIPS & BEST PRACTICES

- Run as non-root ( `1001` ) as runtime images expect
- Set JVM memory with `-XX:MaxRAMPercentage` in containers
- Track EE 12 at `jakarta.ee/specifications` before upgrading

## QUICK SUMMARY

- `thin war` — app only
- `bootable jar` — app + trimmed server
- `env config` — no rebuild per env
- `EE 12` — milestones, 2026–27

# Quick Reference: Annotations



## CDI (11–14)

| ANNOTATION | DOES |
|---|---|
| `@Inject` | inject by type |
| `@ApplicationScoped` | one per app |
| `@RequestScoped` | one per request |
| `@SessionScoped` | one per session |
| `@Dependent` | per injection point |
| `@Named` | EL name |
| `@Produces / @Disposes` | factory / cleanup |
| `@Qualifier` | disambiguate |
| `@Alternative @Priority` | swap impl |
| `@Observes` | event observer |
| `@Interceptor` | interceptor class |

## ENTERPRISE BEANS (08–10)

| ANNOTATION | DOES |
|---|---|
| `@Stateless` | pooled bean |
| `@Singleton` | one instance |
| `@Stateful` | per client |
| `@Startup` | eager singleton |
| `@Lock(READ\|WRITE)` | singleton locking |
| `@Schedule` | cron timer |
| `@Timeout` | timer callback |
| `@Asynchronous` | async method |
| `@MessageDriven` | JMS consumer |
| `@Remove` | end stateful |
| `@TransactionAttribute` | EJB tx type |

## WEB & SERVLET (15–20)

| ANNOTATION | DOES |
|---|---|
| `@WebServlet` | servlet mapping |
| `@WebFilter` | filter mapping |
| `@WebListener` | lifecycle listener |
| `@MultipartConfig` | file uploads |
| `@ServerEndpoint` | WebSocket endpoint |
| `@OnOpen / @OnMessage` | WebSocket callbacks |
| `@FacesConverter` | Faces converter |
| `@ViewScoped` | Faces view scope |
| `@FlowScoped` | Faces flow |
| `@Model` | @Named @RequestScoped |

## REST & JSON (21–24)

| ANNOTATION | DOES |
|---|---|
| `@ApplicationPath` | REST base path |
| `@Path` | resource URI |
| `@GET @POST @PUT @DELETE` | verbs |
| `@Produces / @Consumes` | media types |
| `@PathParam / @QueryParam` | params |
| `@BeanParam` | param holder |
| `@Provider` | register provider |
| `@NameBinding` | targeted filters |
| `@JsonbProperty` | JSON name |
| `@JsonbTransient` | skip field |

## PERSISTENCE & DATA (25–30)

| ANNOTATION | DOES |
|---|---|
| `@Entity / @Table` | mapped class |
| `@Id @GeneratedValue` | primary key |
| `@Column` | column options |
| `@ManyToOne…` | relationships |
| `@JoinColumn` | FK column |
| `@Embeddable` | value objects |
| `@Version` | optimistic lock |
| `@PersistenceContext` | inject EM |
| `@Repository` | Jakarta Data repo |
| `@Find / @Query` | Data queries |

## VALIDATION · SECURITY · TX (30–33)

| ANNOTATION | DOES |
|---|---|
| `@NotNull @NotBlank` | required |
| `@Size @Min @Max` | ranges |
| `@Valid` | cascade / trigger |
| `@RolesAllowed` | role check |
| `@PermitAll / @DenyAll` | open / closed |
| `@DeclareRoles` | declare roles |
| `@Transactional` | CDI transaction |
| `@ManagedExecutorDefinition` | define executor |
| `@ConfigProperty` | MP config value |
| `@Retry @Timeout` | MP fault tolerance |

## ⚠ COMMON MISTAKES

- Spring's `@Transactional` / `@Autowired` in Jakarta code — not processed
- `javax.*` imports on EE 9+ — silently ignored annotations
- `@Named` alone — not a bean under `annotated` discovery

## ✓ TIPS & BEST PRACTICES

- Check the import package first when an annotation "does nothing"
- Prefer CDI annotations over EJB equivalents for new code
- Numbers in table titles point to the detail sheets

## QUICK SUMMARY

- `CDI` — scopes, injection, events
- `EJB` — pools, timers, MDBs
- `REST` — resources and params
- `JPA / Data` — entities and repositories

Sheet 40 EE 11

# Quick Reference: Specs, Versions & Files

EE 11 spec list · versions · Core / Web / Platform · javax → jakarta map · descriptor snippets

## JAKARTA EE 11 SPECIFICATIONS

| SPEC | VERSION | PROFILE |
|---|---|---|
| Annotations | 3.0 | Core |
| CDI (Lite / Full) | 4.1 | Core / Web |
| Dependency Injection | 2.0 | Core |
| Interceptors | 2.2 | Core |
| JSON Binding | 3.0 | Core |
| JSON Processing | 2.1 | Core |
| REST | 4.0 | Core |
| Bean Validation | 3.1 | Web |
| Concurrency | 3.1 | Web |
| Data | 1.0 | Web |
| Expression Language | 6.0 | Web |
| Enterprise Beans Lite | 4.0 | Web |
| Faces | 4.1 | Web |
| Pages | 4.0 | Web |
| Persistence | 3.2 | Web |

## (CONTINUED)

| SPEC | VERSION | PROFILE |
|---|---|---|
| Security | 4.0 | Web |
| Authentication | 3.1 | Web |
| Servlet | 6.1 | Web |
| Standard Tag Library | 3.0 | Web |
| Transactions | 2.0 | Web |
| WebSocket | 2.2 | Web |
| Authorization | 3.0 | Platform |
| Batch | 2.1 | Platform |
| Connectors | 2.1 | Platform |
| Enterprise Beans (full) | 4.0 | Platform |
| Mail | 2.1 | Platform |
| Messaging | 3.1 | Platform |
| Activation | 2.1 | Platform |

## JAVAX → JAKARTA PACKAGES

| OLD | NEW |
|---|---|
| javax.inject | jakarta.inject |
| javax.enterprise | jakarta.enterprise |
| javax.ws.rs | jakarta.ws.rs |
| javax.persistence | jakarta.persistence |
| javax.servlet | jakarta.servlet |
| javax.validation | jakarta.validation |
| javax.faces | jakarta.faces |
| javax.ejb | jakarta.ejb |
| javax.sql (Java SE) | unchanged |

## DESCRIPTOR ROOTS (EE 11)

| FILE | ROOT ELEMENT · VERSION |
|---|---|
| beans.xml | `<beans version="4.1" bean-discovery-mode="annotated">` |
| persistence.xml | `<persistence version="3.2">` |
| web.xml | `<web-app version="6.1">` |
| faces-config.xml | `<faces-config version="4.1">` |
| batch job | `<job version="2.0">` |
| namespace | https://jakarta.ee/xml/ns/jakartaee |
| persistence ns | https://jakarta.ee/xml/ns/persistence |

## ⚠ COMMON MISTAKES

- Mixing descriptor versions — an old `web-app version="4.0"` can switch servers into legacy mode
- Targeting Web Profile APIs on a Core runtime — missing at runtime
- Renaming `javax.sql` / `javax.crypto` — Java SE packages keep `javax`

## ✓ TIPS & BEST PRACTICES

- Match descriptor versions to your EE release
- Check your runtime's certified profile on jakarta.ee/compatibility
- Omit descriptors entirely when annotations suffice

## QUICK SUMMARY

- `Core` — REST, CDI Lite, JSON
- `Web` — + Servlet, JPA, Faces, Security, Data
- `Platform` — + Messaging, Batch, Mail
- `jakarta.*` — every EE package since 9